最新更新: https://dl.acm.org/doi/10.1145/3719027.3765191

研究文章

自动检测在线欺骗模式

ASMIT NAYAK，美国威斯康星大学麦迪逊分校，麦迪逊，威
WANI，美国威斯康星大学麦迪逊分校，麦迪逊，威斯康星州
美国威斯康星大学麦迪逊分校，麦迪逊，威斯康星州，美国
HANDELWAL，美国威斯康星大学麦迪逊分校，麦迪逊，威
FAWAZ，美国威斯康星大学麦迪逊分校，麦迪逊，威斯康星

开放获取支持由：威斯康星大学麦迪逊分校提供

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3719027.3765

RESEARCH-ARTICLE

Automatically Detecting Online Deceptive Patte

ASMIT NAYAK, University of Wisconsin-Madison, Madison, W
YASH WANI, University of Wisconsin-Madison, Madison, WI,
SHIRLEY ZHANG, University of Wisconsin-Madison, Madison
RISHABH KHANDELWAL, University of Wisconsin-Madison,
KASSEM FAWAZ, University of Wisconsin-Madison, Madison,

Open Access Support provided by:
University of Wisconsin-Madison

191

威斯康星州，美国 YASH
州，美国 SHIRLEY ZHANG，
国 RISHABH K
威斯康星州，美国 KASSEM
星州，美国

发布日期：2025年11月22日

CCS '25：ACM SIGSAC 计算机与通
安全会议 2025年10月13日 - 17日
台北

会议赞助商：SIGSAC

65191

tterns

, WI, United States
I, United States
son, WI, United States
on, Madison, WI, United States
n, WI, United States

Published: 22 November 2025

CCS '25: ACM SIGSAC Conference on
Computer and Communications Secu
October 13 - 17, 2025
Taipei, Taiwan

Conference Sponsors:
SIGSAC

自动检测在线欺骗模式

阿斯密特·纳亚克 威斯康星大学麦迪逊分校 美国威斯康星州麦迪逊 anayak6@wisc.edu

Yash Wani* 学麦迪逊分校, 州麦迪逊 ywani

里沙布·坎德瓦尔 威斯康星大学麦迪逊分校 美国威斯康星州麦迪逊 rkhandelwal3@wisc.edu

摘要

数字界面中的欺骗性模式会操纵用户做出非意图的决策，利用认知偏差和心理脆弱性。这些模式已在各种数字平台上无处不在。虽然从法律和技术角度出现了缓解欺骗性模式的努力，但在创建可用且可扩展的解决方案方面仍存在显著差距。我们提出了AutoBot框架以弥补这一差距，并帮助网站利益相关者识别和缓解网络上的欺骗性模式。AutoBot能够准确识别和定位网站截图中的欺骗性模式，而无需依赖底层HTML代码。AutoBot采用两阶段管道，利用专门的视觉模型分析网站截图、识别交互元素并提取文本特征。接着，使用大型语言模型，AutoBot理解这些元素周围的上下文，以确定欺骗性模式的存在。

我们实现了 AutoBot，应用于三个面向不同网络利益相关者的下游应用程序：（1）一个本地浏览器扩展，为用户提供实时反馈，（2）一个 Lighthouse 审计，以告知开发者他们网站上潜在的欺骗性模式，以及（3）作为研究人员和监管机构的测量工具。

CCS 概念

• 以人为本的计算 → 基于网络的交互；人机交互 (HCI)；• 计算方法 → 机器学习；人工智能；• 安全与隐私 → 安全与隐私的可用性；安全与隐私的社会方面。

*两位作者对这项研究的贡献相同。

记录的更正版本。V.1.1。2026年1月2日发布。仍然可以通过S访问VoR

Automatically Detecting O

Asmit Nayak University of Wisconsin-Madison Madison, Wisconsin, USA anayak6@wisc.edu

Yash University of Wisc Madison, Wisc ywani@w

Rishabh Khandelwal University of Wisconsin-Madison Madison, Wisconsin, USA rkhandelwal3@wisc.edu

Abstract

Deceptive patterns in digital interfaces manipulate users into making unintended decisions, exploiting cognitive biases and psychological vulnerabilities. These patterns have become ubiquitous on various digital platforms. While efforts to mitigate deceptive patterns have emerged from legal and technical perspectives, a significant gap remains in creating usable and scalable solutions. We introduce our *AutoBot* framework to address this gap and help web stakeholders navigate and mitigate online deceptive patterns. *AutoBot* accurately identifies and localizes deceptive patterns from a screenshot of a website without relying on the underlying HTML code. *AutoBot* employs a two-stage pipeline that leverages the capabilities of specialized vision models to analyze website screenshots, identify interactive elements, and extract textual features. Next, using a large language model, *AutoBot* understands the context surrounding these elements to determine the presence of deceptive patterns. We also use *AutoBot*, to create a synthetic dataset to distill knowledge from '*teacher*' LLMs to smaller language models. Through extensive evaluation, we demonstrate *AutoBot*'s effectiveness in detecting deceptive patterns on the web, achieving an F1-score of 0.93 in this task, underscoring its potential as an essential tool for mitigating online deceptive patterns.

We implement *AutoBot*, across three downstream applications targeting different web stakeholders: (1) a local browser extension providing users with real-time feedback, (2) a Lighthouse audit to inform developers of potential deceptive patterns on their sites, and (3) as a measurement tool for researchers and regulators.

CCS Concepts

• Human-centered computing → Web-based interaction; **Human computer interaction (HCI)**; **• Computing methodologies → Machine learning**; **Artificial intelligence**; **• Security and privacy →** Usability in security and privacy; *Social aspects of security and privacy*.

*Both authors contributed equally to this research.

CCS '25, Taipei, Taiwan

ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765191

Corrected Version of Record. V.1.1. Published January 2, 2026. The VoR may still be accessed

ni* 威斯康星大
，美国威斯康星
ni@wisc.edu

张雪莉* 威斯康星大学麦迪
逊分校 美国威斯康星州麦迪逊
hzhang664@wisc.edu

卡塞姆·法瓦兹
威斯康星大学麦迪逊分校 美国威斯
康星州麦迪逊 kfawaz@wisc.edu

关键词
欺骗性模式；自动检测欺骗性模式；多模态大型语言模型；
计算机视觉；知识蒸馏；合成数据生成；UI 元素检测

ACM 参考格式：
Asmit Nayak, Yash Wani, Shirley Zhang, Rishabh Khandelwal 和
Kassem Fawaz. 2025. 自动检测在线欺骗模式。载于《2025 年 ACM SIGSAC
计算机与通信安全会议（CCS ’ 25）论文集》，2025 年 10 月 13–17 日，台湾台
北。美国纽约：ACM，15 页。https://doi.org/10.1145/3719027.3765191

1 引言

欺骗性模式，也称为“暗黑模式”，是通过设计选择来操控用户在使用应用时做出非预期决策的方式。这些模式利用认知偏差和心理弱点来影响用户行为，通常以对用户不利、对服务提供者有利的方式进行。这些欺骗性模式的日益增长对各种在线活动的用户体验质量产生负面影响，例如在线购物、使用社交媒体、玩电子游戏或仅仅浏览网页。这些模式的广泛存在有充分的记录，例如 Harry Brignull 的 deceptive.design 资源以及 Caroline Sliders 在 The Pudding 的分析中都有显著的案例。

尽管近期在用户意识、媒体曝光和隐私法规的推动下，网页体验有所改善，但欺骗性模式仍然带来了重大挑战。因此，用户面临的风险包括财务损失、隐私侵犯以及对易受伤害人群（包括儿童）的剥削。为此，研究人员探索了在网页上检测和分类欺骗性模式的不同方法。早期的努力包括通过人工分析来研究互联网上欺骗性模式的分布。这类方法由于网站数量庞大、动态变化性强以及界面多样性，在大规模下不可行。最近的努力包括基于启发式和机器学习的方法。然而，正如我们后续将展示的，这些方法在识别实际环境中的欺骗性模式时往往准确性有限。

认识到当前方法的局限性，迫切需要自动化工具来协助网站利益相关者

[1]https://www.deceptive.design/[2]
https://pudding.cool/2023/05/dark-patterns/

Online Deceptive Patterns

Wani*
isconsin-Madison
isconsin, USA
wisc.edu

Shirley Zhang*
University of Wisconsin-Madison
Madison, Wisconsin, USA
hzhang664@wisc.edu

Kassem Fawaz
University of Wisconsin-Madison
Madison, Wisconsin, USA
kfawaz@wisc.edu

Keywords
Deceptive Patterns; Automated Detection of Deceptive Patterns; Multi-Modal Large Language Models; Computer Vision; Knowledge Distillation; Synthetic Data Generation; UI Element Detection

ACM Reference Format:
Asmit Nayak, Yash Wani, Shirley Zhang, Rishabh Khandelwal, and Kassem Fawaz. 2025. Automatically Detecting Online Deceptive Patterns. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3765191

1 Introduction

Deceptive patterns, also known as '*Dark Patterns*,' are design choices that manipulate users into making unintended decisions as they interact with applications. These patterns exploit cognitive biases and psychological vulnerabilities to influence user behavior, often in ways that benefit the service provider at the user's expense [10, 11]. The growing use of deceptive patterns negatively impacts the quality of user experiences across various online activities, such as online purchases, engaging with social media, playing video games, or simply browsing the web [43]. The widespread nature of these patterns is well documented, with notable examples from resources like Harry Brignull's `deceptive.design`[1] and Caroline Sliders' analysis at `The Pudding`[2].

Despite the recent push toward better web experience, fueled by user awareness, media revelations, and privacy regulations [1, 15], deceptive patterns continue to pose substantial challenges [35]. As a result, users are at risk of harm, such as financial loss [58], privacy violations [6], and the exploitation of vulnerable populations, including children [65]. In response, researchers have explored different approaches to detect and classify deceptive patterns on the web. Earlier efforts included manual analysis to analyze the distribution of deceptive patterns on the Internet [10, 26, 43]. Such approaches are infeasible at scale due to the sheer volume, dynamic nature, and diversity of website interfaces. More recent efforts include heuristic- and ML-based methods [4, 10, 16, 26, 42, 43, 52, 64, 72]. However, these approaches often exhibit limited accuracy when identifying deceptive patterns in the wild, as we show later.

Recognizing the limitations of current approaches, there is a pressing need for automated tools to assist web stakeholders in

[1]https://www.deceptive.design/
[2]https://pudding.cool/2023/05/dark-patterns/

图1：OpenAI 的 GPT4.5 错误地将“全部拒绝”按钮的颜色识别为比其他按钮不显眼，从而错误地分类为“提示”。同样，Gemini 2.5 Pro 错误地认为 cookie 通知中的两个按钮在视觉上彼此不同，导致将其错误分类为“提示”。

导航和减轻在线欺骗模式。这种自动化工具必须执行两个主要任务：1）准确识别欺骗模式，2）精确定位它们在网站中的位置。能够同时识别和定位这些模式为网站相关方带来了若干好处。首先，网站用户可以被提醒他们访问的网站中存在的欺骗模式，从而做出明智的决策。其次，监管机构可以利用这样的工具在大规模上识别欺骗模式，促进执法和政策制定。第三，开发人员可以深入了解其网站中可能存在问题的元素，推动更具道德的设计实践 [66]。

我们提出了一个自动化欺骗模式检测框架——AutoBot，以应对这些限制。AutoBot能够准确识别并定位网页截图中的欺骗模式。它不依赖网页的底层HTML代码，因为HTML代码相比截图往往不够稳定。HTML的实现和代码在不同网页甚至不同访问情况下可能差异很大，而截图和文本则相对更为一致 [31]。AutoBot采用模块化设计，将任务分解为两个独立模块，并为每个模块利用现有的最先进模型。具体来说，AutoBot使用视觉模块，该模块分析截图以准确定位UI元素，并将关键特征提取为结构化的纯文本格式（ElementMap）。然后，它将这个ElementMap输入语言模块，该模块使用大型语言模型（LLM）分析ElementMap，并根据定义的分类法（第2.1节）分配欺骗模式。

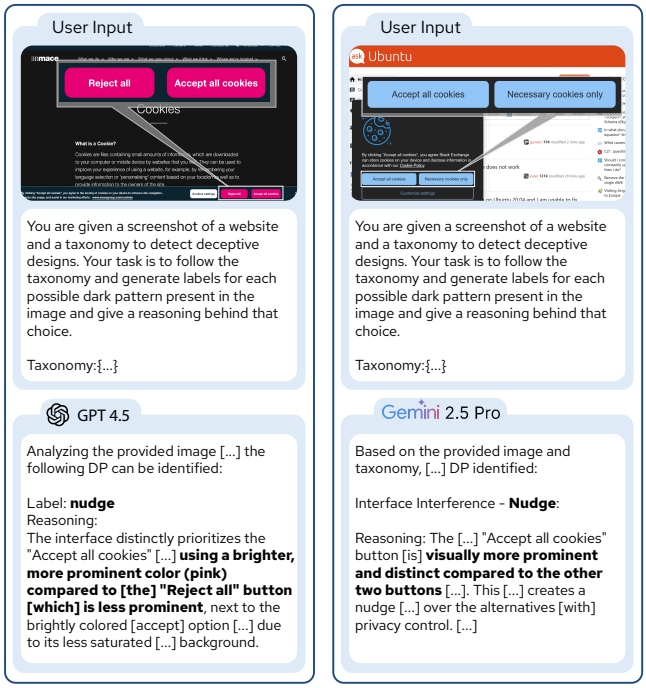


Figure 1: OpenAI's `GPT4.5` incorrectly identifies the color of the "Reject All" button as being less prominent than the other, leading to an incorrect classification of "nudge". Similarly, `Gemini 2.5 Pro` incorrectly notes that the two buttons in the cookie notices are visually distinct from each other, resulting in a misclassification of "nudge".

navigating and mitigating online deceptive patterns. Such an automated tool must perform two primary tasks: 1) accurately identify deceptive patterns and 2) precisely *localize* their position within the website. The ability to both identify and localize these patterns offers several benefits to web stakeholders. First, web users can be alerted to deceptive patterns on websites they visit, enabling informed decision-making. Second, regulators can leverage such tools to identify deceptive patterns at scale, facilitating enforcement and policy development. Third, developers can gain insights into potentially problematic elements within their websites, promoting more ethical design practices [66].

We propose an automated deceptive pattern detection framework, *AutoBot*, to address these limitations. *AutoBot* accurately identifies and localizes the deceptive patterns from a screenshot of a webpage. It does not rely on the underlying `HTML` code of the webpage, which tends to be less stable than screenshots. `HTML` implementation and code can vary significantly across webpages and even different accesses, while screenshots and text remain more consistent [31]. *AutoBot* adopts a modular design, breaking down the task into two distinct modules and leveraging existing state-of-the-art models for each. Specifically, *AutoBot* utilizes a *Vision Module*, which analyzes screenshots to accurately localize UI elements, extracting essential features into a structured, text-only format (*ElementMap*). It feeds this *ElementMap* to a *Language Module* that employs a Large Language Model (LLM) to analyze the *ElementMap* and assign a deceptive pattern based on a defined taxonomy (Section 2.1).

AutoBot 的设计避免了直接提示视觉大型语言模型（VLLM）进行端到端分析的陷阱。如我们在图 1 中所示，这些模型已知会产生幻觉，导致误报，从而破坏其可靠性 [60]。此外，正如最近的研究 [17, 78] 和第 4.4.2 节所示，VLLM 目前缺乏对 UI 元素进行准确定位的能力。虽然微调 VLLM 可能会提升性能，但对大规模标注数据集和大量计算资源的巨大需求使这种方法不切实际 [7]。

AutoBot 利用专门视觉模型的能力来协助定位任务，并使用大语言模型（LLM）进行精准的欺骗模式识别。虽然 LLM 在检测欺骗模式方面表现出强大性能（见第 5.3 节），但由于成本、延迟和隐私问题，大规模使用这些 LLM 可能是不可行的。在本工作中，我们展示了如何使用 LLM 作为“教师”创建一个合成数据集，并将其知识蒸馏到较小的语言模型（SLM），如 Qwen2.5-1.5B，以及非常小的语言模型（vSLM），如 Flan-T5。我们在第 5.3 节中展示了对这些模型的详细评估。

我们展示了 AutoBot 在三种实例中的实际应用，面向不同的网络利益相关者。首先，我们设计并实现了一个浏览器扩展（第 7.1 节），使用 AutoBot 自动检测并突出显示网站上的欺骗性模式，为个人电脑用户提供实时帮助。其次，我们创建了一个自定义的 Lighthouse 审核（第 7.2 节），利用 AutoBot 通知开发者其网站上可能存在的欺骗性模式，直接整合到开发者工作流中，并提供可量化的评分。第三，我们展示了研究人员和监管机构如何使用 AutoBot 对在线欺骗性模式进行大规模测量和分析（第 7.3 节）。我们对超过 11,000 个流行（Tranco）和电子商务（Shopify）网站的测量显示了欺骗性模式的普遍存在，许多网站在单个网页上显示多种模式。

2 背景与相关工作

网页欺诈模式是指网站界面设计选择，用来操纵或欺骗用户，使其做出可能不会做出的决定 [10]。此类模式的示例包括隐藏费用、强制续订、电子商务网站中的误导行为，以及社交媒体平台中侵犯隐私的默认设置。在这里，我们提出了一个经过筛选的分类法，根据现有研究对网页欺诈模式进行分类。我们还调查了现有关于检测网站欺诈模式的研究工作。

2.1 欺骗模式的分类

Brignull 等人在 2010 [10] 中提出了首个欺骗性模式分类。Conti 等人将该分类扩展为包括“恶意界面设计” [19]。Bösch 等人 [9] 引入了一个类似的分类，称为“隐私黑暗模式”，其中包括更多以隐私为中心的类别，如“强制注册”和“隐藏法律条款”。

最近，Gray 等人 [26] 创建了一个统一的语料库，用于检测用户界面中的欺骗性设计，基于之前的分类法和类别。从那时起，各种工作进一步将该分类法适用于特定领域。例如，Lewis

AutoBot's design avoids the pitfalls of directly prompting Vision Large Language Model (VLLM) for end-to-end analysis. These models, as we show in Figure 1, are known to hallucinate, leading to false positives undermining their reliability [60]. Furthermore, VLLMs currently lack the capability for accurate localization of UI elements, as shown in recent works [17, 78] and in Section 4.4.2. While fine-tuning VLLMs could potentially improve performance, the substantial demand for large annotated datasets and significant computing resources renders this approach impractical [7].

AutoBot leverages the capabilities of specialized vision models to help with the localization task and utilizes LLMs to perform accurate deceptive pattern identification. While LLMs have shown strong performance in detecting deceptive patterns (see Section 5.3), large-scale use of these LLMs might be prohibitive due to cost, latency, and privacy concerns. In this work, we show how we can create a synthetic dataset using an LLM as the '*teacher*' and distill its knowledge into smaller language models (SLMs), like `Qwen2.5-1.5B`, and very small language models (vSLMs), like `Flan-T5`. We show a detailed evaluation of these models in Section 5.3.

We demonstrate the practical applications of *AutoBot* in three instantiations, targeting different web stakeholders. First, we design, and implement a browser extension (Section 7.1) using *AutoBot* to automatically detect and highlight deceptive patterns on websites, providing real-time user assistance on personal computers. Second, we create a custom Lighthouse audit (Section 7.2) that leverages *AutoBot* to inform developers of potential deceptive patterns on their sites, integrating directly into developer workflows and providing a quantifiable score. Third, we demonstrate how researchers and regulators can use *AutoBot* to perform a large-scale measurement and analysis of the online deceptive patterns landscape (Section 7.3). Our measurement on over 11,000 websites across popular (Tranco) and e-commerce (Shopify) domains highlighted the prevalence of deceptive patterns, with many websites exhibiting several patterns on a single webpage.

2 Background and Related Works

Web deceptive patterns refer to website interface design choices that manipulate or deceive users into making decisions they might not otherwise make [10]. Examples of such patterns include hidden costs, forced continuity, misdirection in e-commerce websites, and privacy-invasive default settings in social media platforms. Here, we present a filtered taxonomy to categorize web deceptive patterns based on existing work. We also survey existing works on detecting deceptive patterns on websites.

2.1 Taxonomy for Deceptive Patterns

Brignull et al. presented the first taxonomy of deceptive patterns in 2010 [10]. Conti et al. expanded this taxonomy to include 'malicious interface designs' [19]. Bösch et al. [9] introduced a similar taxonomy called 'privacy dark patterns', which included more privacy-centric categories such as 'Forced Registration' and 'Hidden Legalese Stipulations.'

More recently, Gray et al. [26] created a unified corpus to detect deceptive designs in user interfaces, building on previous taxonomies and categories. Since then, various works have further adapted the taxonomy for specific domains. For instance, Lewis

图 2：欺骗模式分类。AutoBot 将文本元素分类为五类高级欺骗模式：界面干扰、阻碍、强制操作、潜行以及非欺骗性。

et al. [36] 编纂了针对移动应用和游戏的欺骗模式，而 Mathur et al. [43] 将该分类法调整为关注购物网站中存在的欺骗模式。Chen et al. [16] 和 Mansur et al. [42] 的工作将该分类法扩展到检测移动和网络应用中的欺骗模式。尽管先前的研究开发了各种分类法来对欺骗模式进行分类，但这些努力是不一致的，并且通常是特定领域的。为了解决这些问题，Gray et al. [27] 引入了一种统一的欺骗模式本体，将过去的文献整合到监管报告和学术研究中。

2.1.1 过滤后的分类法。由于我们的工作，如文献中的其他研究一样，侧重于在页面层面检测欺骗性模式，因此排除了那些需要系统理解用户跨多个页面在一段时间内的操作的模式，例如“偷偷加入购物车”——即物品被偷偷加入用户的购物车中。

我们将欺骗性模式分为静态（在页面渲染时可见）或动态（需要用户交互或基于时间的触发，例如，“反复提醒”、“难以取消”）。由于我们的系统使用屏幕截图，我们的分析仅限于静态模式，如图2所示。检测动态模式需要时间分析，这超出了本工作的范围。然而，我们的系统为未来通过分析网页代码、时间活动并执行操作来检测动态模式的系统提供了一个构建模块。

我们的分类法是通过筛选 Gray 等人的本体论 [27] 创建的，以聚焦于静态欺骗模式。结果是一个包含四个高级类别和 11 个子类型的分类法，如图 2 所示。在我们的分析中，我们使用了这些子类型的文本描述，这些描述改编自 Brignull 等人 [10],，用于提示语言模型，如后文所示。本工作的完整分类法及其与 Gray 等人工作的映射在本工作的扩展报告中提供 [46]。

2.2 识别欺骗模式

研究人员开发了几种机制来检测和衡量网络欺骗模式的普遍性 [4, 10, 16, 26, 42, 43, 52, 64, 72]。

Curley 等人对早期的手动、自动或半自动检测机制进行了分类 [22]。

2.2.1 基于人工的手动标注。早期识别网络欺骗模式的努力依赖于人工探索。Brignull 等人 [10] 手动浏览网页以编制“耻辱殿堂”：一个包含具有欺骗模式的网站列表。Gray 等人 [26] 扩展了

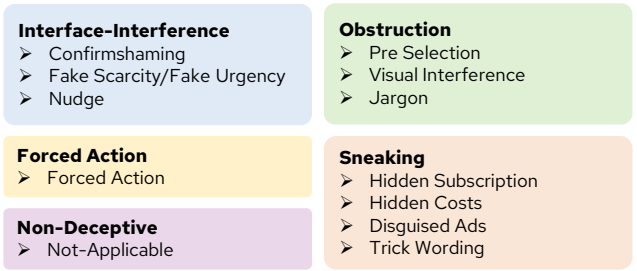


Figure 2: Taxonomy of Deceptive Patterns. *AutoBot* classifies text elements into five high-level deceptive pattern categories: Interface Interference, Obstruction, Forced Action, Sneaking, and Non-Deceptive.

et al. [36] codified deceptive patterns for mobile apps and games, while Mathur et al. [43] adapted the taxonomy to focus on deceptive patterns present in shopping websites. Work by Chen et al. [16] and Mansur et al. [42] extended the taxonomy to detect deceptive patterns in mobile and web apps. Although prior works developed various taxonomies to classify deceptive patterns, these efforts are inconsistent and often domain-specific. To address these issues, Gray et al. [27] introduced a unified ontology of deceptive patterns integrating past literature across regulatory reports and academic works.

2.1.1 Filtered Taxonomy. Since our work, like others in literature, focuses on detecting deceptive patterns at the page level, it excludes patterns that require the system to understand user action across multiple pages over a period of time, such as ‘*Sneak into Basket*’ – where items are sneakily added to users’ carts.

We classify deceptive patterns as either static (visible on page render) or dynamic (requiring user interaction or time-based triggers, e.g., ‘Nagging,’ ‘Hard-to-Cancel’). Since our system uses screenshots, our analysis is limited to static patterns, as defined in Figure 2. Detecting dynamic patterns requires temporal analysis, which is out of scope for this work. However, our system presents a building block for future systems to detect dynamic patterns by analyzing webpage code, temporal activities, and performing actions.

Our taxonomy was created by filtering Gray et al.’s ontology [27] to focus on static deceptive patterns. The result is a taxonomy with four high-level categories and 11 subtypes, as shown in Figure 2. For our analysis, we utilize the textual descriptions of these subtypes, adapted from Brignull et al. [10], to prompt language models, as shown later. The complete taxonomy and its mapping to Gray et al.’s work are provided in the extended report of this work [46].

2.2 Detecting Deceptive Patterns

Researchers developed several mechanisms to detect and measure the prevalence of online deceptive patterns [4, 10, 16, 26, 42, 43, 52, 64, 72]. Curley et al. categorized earlier manual, automated, or semi-automated detection mechanisms [22].

2.2.1 Human-based Manual Annotations. Early efforts to identify online deceptive patterns relied on manual exploration. Brignull et al. [10] manually explored the web to compile a “Hall of Shame”: a list of websites with deceptive patterns. Gray et al. [26] expanded

98

通过在社交媒体上对突出显示具有欺骗性模式的网站的帖子进行关键词搜索，从而构建了这个语料库，并随后进行了人工验证。虽然这种方法高度准确，但由于需要人工验证，因此仅限于领域专家，且缺乏可扩展性。为了加快人工探索的过程，Mathur 等人 [43] 提出了一个基于聚类的流程，根据网站的文本内容将相似的网站分组，然后进行人工检查。虽然该过程加快了数据收集，但仍然缺乏可扩展性。

2.2.2 概率文本与图像模型。尝试自动化人工检查过程的工作包括 Tung 等人的朴素贝叶斯分类器 [72], 、Soe 等人用于标记展示欺骗模式文本的梯度提升树 [64], 以及 Adorna 等人结合朴素贝叶斯分类器和 VGG-19 模型来识别 Cookie 通知中的欺骗性设计 [4]。然而，这些工作通常是特定领域的，无法轻易推广到新的领域。此外，大多数工作仅依赖基于文本的分类器或启发式方法，这限制了其检测视觉欺骗模式（例如基于颜色或文字技巧的模式）的能力。

2.2.3 启发式方法。Chen 等人、Mansur 等人和 Raju 等人的最新研究 [16, 42, 52] 关注于移动应用中欺骗性模式的自动检测。Chen 等人 [16] 和 Mansur 等人 [42] 使用了预定义的启发式方法，使得检测仅限于较简单的欺骗性模式，如伪装广告。先前的研究使用网站的 HTML 代码来识别和检测欺骗性模式。例如 Raju 等人 [52] 采用基于规则的源代码分析来检测广告、强制操作和烦扰等模式 [52]。然而，由于 HTML 的多功能性和开放性这项任务被证明非常具有挑战性。虽然 HTML 的实现和代码在不同网页之间甚至同一网页的不同访问中存在显著差异，但截图和文本则保持更一致。因此，最近的方法已经完全转向使用网站的文本或截图来检测和识别欺骗性设计。

2.2.4 经典机器学习模型。在更广泛的隐私、安全和保安（PSS）领域，研究人员已经开发了工具来帮助用户识别网站上的特定欺骗性设计。例如，Khandelwal 等人的研究 [31, 32] 使用户能够查找和调整隐私设置，并自动禁用非必要的cookie。类似地，Kumar 等人的 OptOutEasy [8] 会自动从隐私政策中查找退出链接，并呈现给用户。这些特定领域的方法不容易应用于欺骗性模式检测，因为它们需要针对每种欺骗性模式重新训练。

2.2.5 大型语言模型 (VLLMs)。先前的研究探索了使用大型语言模型（LLMs）来检测欺骗性模式。Sazid 等人 [56] 使用 GPT-3.5-Turbo 检测欺骗文本，准确率为 92.57%，尽管他们的方法仅限于单行文本，仅涉及 7 种模式，并且忽略了周围的视觉或文本环境。类似地，Schäfer 等人 [57] 利用 GPT-4o 从合成 HTML 中移除欺骗性元素，在 91% 的情况下降低了操控性。然而，由于 HTML 的多样性和非标准化特性 [67] 以及将膨胀的真实世界网站源代码放入 LLM 上下文窗口的挑战，这一方法存在一定的局限性。

最近，与我们的工作同时进行的一项研究，Shi 等人 [60], 引入了 DPGuard 来自动检测欺骗模式

this corpus by performing keyword searches on social media for posts highlighting websites with deceptive patterns, which were then manually validated. While highly accurate, this methodology is limited to domain experts and lacks scalability due to manual validation requirements. To expedite the manual exploration process, Mathur et al. [43] proposed a clustering-based pipeline to group similar websites based on their text content, which is then manually inspected. Although this process accelerates data collection, it still lacks scalability.

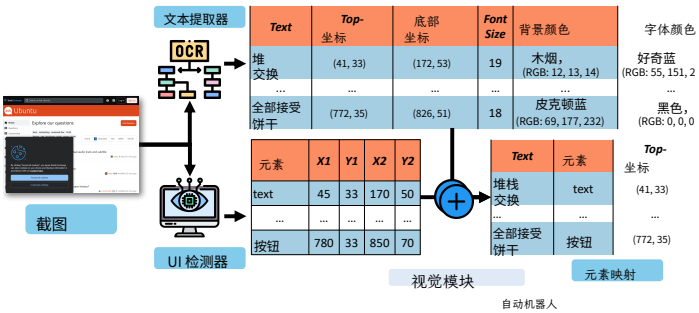
2.2.2 Probabilistic Text & Image Models. Attempts to automate the manual inspection process include Tung et al.'s *Naive Bayes* classifier [72], Soe et al.'s gradient-boosted tree to flag texts showcasing deceptive patterns [64], and Adorna et al.'s combined *Naive Bayes* classifier and `VGG-19` model to identify deceptive designs in cookie notices [4]. However, these works are often domain-specific and cannot readily generalize to new domains. Additionally, most works rely solely on text-based classifiers or heuristics, limiting their ability to detect visual deceptive patterns, such as those based on colors or trick wording.

2.2.3 Heuristic Based Methods. Recent works by Chen et al., Mansur et al., and Raju et al. [16, 42, 52] focus on automated detection of deceptive patterns in mobile apps. Chen et al. [16] and Mansur et al. [42] used predefined heuristics, limiting detection to simpler deceptive patterns like disguised ads. Prior works used the `HTML` code of websites to identify and detect deceptive patterns. For example, Raju et al. [52] employed rule-based source code analysis to detect patterns like ads, forced action, and nagging [52]. However, due to the versatile and open nature of `HTML`, this task proves to be very challenging. While `HTML` implementation and code vary significantly across webpages, and even across different accesses of the same page, screenshots and text remain more consistent. As a result, recent approaches have moved completely towards text or screenshots of websites to detect and identify deceptive designs.

2.2.4 Classical ML Models. In the broader domain of Privacy, Safety, and Security (PSS), researchers have developed tools to help users navigate specific deceptive designs on websites. For instance, works by Khandelwal et al. [31, 32] enable users to find and adjust privacy settings and automatically disable non-essential cookies. Similarly, OptOutEasy by Kumar et al. [8] automatically finds opt-out links from privacy policies and surfaces them to users. These domain-specific approaches do not readily apply to deceptive pattern detection as they require retraining for each deceptive pattern.

2.2.5 Large Language Models (VLLMs). Prior work have explored using LLMs to detect deceptive patterns. Sazid et al. [56] used GPT-3.5-Turbo to detect deceptive text with 92.57% accuracy, athough their method is limited to singular text lines, only 7 types of patterns, and ignores any surrounding visual or textual context. Similarly, Schäfer et al. [57] utilized GPT-4o to remove deceptive elements from synthetic HTML, reducing manipulativeness in 91% of cases. This approach, however, is constrained due to the versatile and non-standardized nature of HTML [67] and the challenge of fitting inflated, real-world website source code into an LLM's context window.

More recently, a concurrent work with ours, Shi et al. [60], introduces *DPGuard* to automatically detect deceptive patterns from



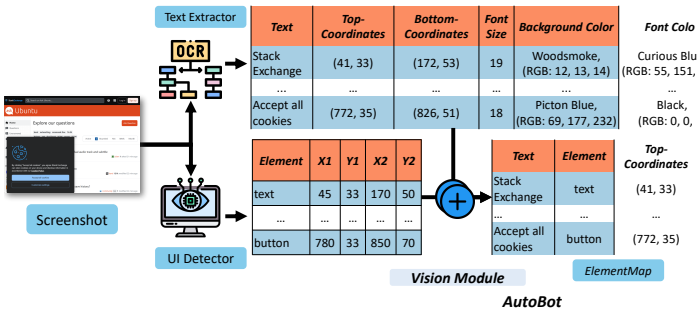


图 3：AutoBot 工作流程概览。AutoBot 通过包含视觉模块和语言模块的类型及推理。然后，这些结果被浏览器扩展程序和 Lighthouse 等应用程

Figure 3: Overview of *AutoBot*'s working process. *AutoBot* takes a screens and the Language Module, and returns the Deceptive Pattern classificat results are then used by applications such as browser extensions and Lig

通过直接提示 VLLMs 截取移动应用程序和网站的屏幕截图。我们在第 3 节后面展示了，VLLMs 在从屏幕截图中检测欺骗性模式方面表现不佳，并且容易出现幻觉和误报。此外，DPGuard 仅检测这些欺骗性模式的存在，而不提供任何位置参考。我们的工作以显著更高的准确率识别欺骗性模式并提取其位置，使我们能够向用户准确展示这些模式出现的位置。

3 系统概述

这项工作提出了 AutoBot，一个端到端框架，用于识别欺骗模式并提取它们在网页上的位置。在接收到网页截图后，它会识别页面上的 UI 元素，并将识别出的元素及相关文本输入到大型语言模型（LLM）中。输出是每个元素与第 2.1 节中对应欺骗模式的映射。

为什么要使用截图？之前分析网站的研究主要依赖 HTML 分析。这种方法面临重大挑战，因为网站具有动态特性。网站越来越多地使用能够实时修改文档对象模型（DOM）的 JavaScript 框架，使得静态 HTML 分析不足以应对。此外，编码实践的多样性和混淆技术为基于 HTML 的分析增加了额外困难。为了解决这些挑战，AutoBot 采用了一种新方法，专注于网站的不变方面：用户体验。通过利用视觉信号和相关文本，AutoBot 模拟用户如何感知和与网站交互。这种方法有几个优点：(1) 它对底层技术的变化具有韧性，因为它捕捉的是实际呈现的内容。(2) 它使我们能够分析用户所遇到的相同信息，从而更准确地呈现潜在的欺骗性模式。

视觉大型语言模型（VLLMs）。VLLMs的最新进展为分析截图并通过适当提示突出模式提供了途径。为此，我们对GPT-4.5 [3] 和Gemini 2.5 Pro [68] 在检测网页欺骗模式方面的有效性进行了实证评估。我们的实验表明，虽然这些模型可以检测欺骗模式，但它们经常出现幻觉并给出误报答案。如图1所示，Gemini 2.5 Pro和GPT-4.5在识别欺骗模式方面都存在困难。

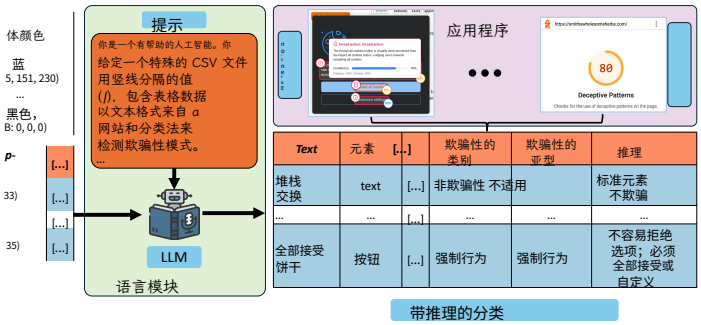
screenshots of mobile apps and websites by directly prompting VLLMs. We show later in Section 3 that VLLMs perform poorly in detecting deceptive patterns from screenshots and are often prone to hallucinations and false positives. Additionally, *DPGuard* only detects the presence of these deceptive patterns without any positional reference. Our work identifies deceptive patterns with significantly higher accuracy and extracts their positions, enabling us to show users exactly where these patterns occur.

3 System Overview

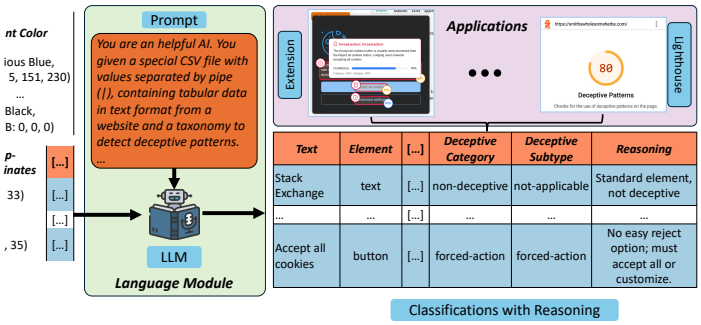
This work presents *AutoBot*, an end-to-end framework that identifies the deceptive patterns and extracts their positions on a webpage. After receiving a screenshot of a webpage, it identifies UI elements on the page and feeds the identified elements along with the associated text to an LLM. The output is a mapping of each element to a corresponding deceptive pattern from Section 2.1.

Why Screenshots? Prior works analyzing websites [31, 32] have primarily relied on HTML analysis. This approach faces significant challenges due to the dynamic nature of websites. Websites increasingly employ *JavaScript* frameworks that modify the Document Object Model (DOM) on the fly, rendering static HTML analysis insufficient. Furthermore, the diversity in the coding practices and obfuscation techniques provide additional challenges in HTML-based analyses. To address these challenges, *AutoBot* adopts a novel approach focusing on the invariant aspect of websites: *the user experience*. By leveraging the visual signals and associated text, *AutoBot* models how users perceive and interact with websites. This approach offers several advantages: (1) It is resilient to change in underlying technologies as it captures the actual rendered content. (2) It allows us to analyze the same information that the user encounters, providing a more accurate representation of the potential deceptive patterns.

Vision Large Language Models (VLLMs). Recent advances in VLLMs offer a venue for analyzing screenshots and highlighting patterns with proper prompting. To this end, we empirically evaluate the effectiveness of `GPT4.5` [3] and `Gemini 2.5 Pro` [68] in detecting web deceptive patterns. Our experiments showed that while these models can detect deceptive patterns, but they often hallucinate and give false positive answers. As shown in Figure 1, both `Gemini 2.5 Pro` and `GPT-4.5` struggle to identify the deceptive



块的多阶段框架截取屏幕截图，并返回每个截图元素的欺骗模式分类、子程序使用。



enshot through a multi-stage framework consisting of the Vision Module cation, Subtype, and reasoning for each element in the screenshot. The Lighthouse.

屏幕截图中的模式。Shi 等人最近在该领域的工作 [60], DPGuard，直接使用这些模型来检测，而不是定位欺骗性模式。与 Shi 等人的评估一致，我们发现 DPGuard 面临性能问题，在各种网站上难以泛化（参见第 6 节），在网站上检测欺骗性模式时仅取得 0.3452 的宏观得分。

此外，VLLMs 经常在输入图像中精确定位元素时遇到困难 [17, 37, 78],，并且无法开箱即用地定位欺骗性模式。然而，当提供边界框坐标时，这些模型可以有效地推理对象的空间排列 [61]。

模块。AutoBot 利用上述洞察自动检测并定位网站上的欺骗性模式，如图 3 所示。它将定位问题和欺骗性模式检测分为两个任务：视觉和语言。这种分解使 AutoBot 能够独立利用视觉技术精确定位元素，并利用强大的语言模型准确分类模式。

(1) 视觉模块：视觉模块将网页截图映射为如图3所示的元素表我们将这种表格表示称为 ElementMap。ElementMap 包含与元素相关的文本及其其他特征：元素类型、边界框坐标、字体大小、背景颜色和字体颜色。该模块通过解析网页截图来解决高误报率和定位问题。

(2) 语言模块：语言模块（第5节）以 ElementMap 作为输入，并将每个元素映射到第2.1节分类法中的一个欺骗模式。该模块根据每个元素的空间上下文和视觉特征进行推理。我们探讨了该模块的不同实例化方法，这些方法在成本、训练需求和准确性方面具有不同的权衡。

4视觉模块

，视觉模块通过以下三个步骤从网站截图生成元素地图，如图3所示。

(1) 文本提取：从截图中提取文本、文本的边界框以及相关特征。

patterns in screenshots. A recent work in this domain by Shi et al. [60], *DPGuard*, uses these models directly to detect, not localize, deceptive patterns. Consistent with Shi et al.'s evaluation, we find that *DPGuard* faces performance issues, struggling to generalize over a variety of websites (refer to Section 6), achieving only a macro score of `0.3452` in detecting deceptive patterns on websites.

In addition, VLLMs frequently struggle with the precise location of elements in an input image [17, 37, 78], and cannot be used to localize deceptive patterns out of the box. However, when given bounding box coordinates, these models can effectively reason about the spatial arrangements of objects [61].

Modules. AutoBot leverages the above insights to automatically detect and localize deceptive patterns on websites, as shown in Figure 3. It breaks the localization problem and deceptive pattern detection into two tasks: vision and language. This breakdown allows *AutoBot* to independently leverage vision techniques for the precise localization of elements and powerful language models for the accurate classification of patterns.

(1) **Vision Module:** The *Vision Module* maps a screenshot of a webpage to a table of elements as shown in Figure 3. We refer to this tabular representation as *ElementMap*. The *ElementMa* contains the text associated with the element along with its other features: element type, bounding box coordinates, font size, background color, and font color. This module addresses the issues of high false positive rates and localization by parsing the screenshot of a webpage.

(2) **Language Module:** The *Language Module* (Section 5) takes the *ElementMap* as input and maps each element to a deceptive pattern from the taxonomy in Section 2.1. This module reasons about each element considering its spatial context and visual features. We explore different instantiations of this module with different trade-offs in terms of cost, need for training, and accuracy.

s

4 Vision Module

The *Vision Module* generates an *ElementMap* from a screenshot of the website through the following three steps, as shown in Figure 3.

(1) **Text Extraction:** Extract the text, the bounding boxes of the text, and the associated features from the screenshot.

(2) Web-UI 元素检测：定位 UI 元素，提取它们的边界框，并从屏幕截图中识别它们的类型。

(3) 元素映射生成：将上述步骤的结果合并为网站的表格表示，以生成元素映射。

4.1 文本提取

文本提取步骤首先对网站截图执行光学字符识别（OCR）。我们使用 Google Vision OCR API [3]，因为它在从不同规模和分辨率的图像中提取文本方面具有高准确性。该 API 返回检测到的文本块。我们根据文本块的接近性将它们连接，以形成连贯的文本块。这些文本块是带有各自文本内容的边界框列表。然后，我们检索每个边界框的字体大小、字体颜色和背景颜色，如图 3 所示。我们通过从边界框的顶部 y 坐标减去底部 y 坐标来计算字体大小。为了确定颜色信息，我们使用 extcolors [14] 包，提取最显著的颜色（背景）和第二显著的颜色（字体）

这种方法解决了Soe等人 [64] 在以往机器学习方法中指出的一个关键限制：缺乏对用户所感知的界面丰富性的表示，例如文本布局以及字体与背景颜色之间的对比。我们的方法捕捉了这些详细的文本特征，以及网站上文本元素的相对位置，从而提供了用户体验文本内容的全面表示。

4.2 Web-UI元素检测

Web-UI 元素检测步骤使用相同的截图来识别并定位这7个Web界面元素：按钮、复选框（☑, ■）、单选按钮（●, ○）和切换开关（◐, ◑）。此步骤为提取的文本提供上下文，并使对网页结构的理解更全面。例如，区分可点击的按钮和静态文本块可能非常重要，因为看似简单的文本在被识别为按钮时可能是误导性的操作提示。同样，识别出的复选框状态（选中或未选中）可以揭示用户可能忽略的预选项。

在下文中，我们描述了如何调查现有的网页用户界面（Web-UI）检测方法及其基础数据集。由于我们发现这些方法和数据集不适用于欺骗性模式检测任务，我们描述了如何使用 YOLOv10 在自定义数据集上训练实时 Web-UI 元素检测器。

现有网页界面检测方法的局限性。尽管网页界面元素检测是一个研究较多的问题，但现有方法在我们的欺骗模式检测任务中存在若干局限。像 Ominparser *2* [40],、Ferret UI *2* [38],、UEID [76], 和 Element Detector [75] 这样的检测器无法区分已选中和未选中状态。像 UISketch [59] 这样的检测器在真实网站上的表现较差。ScreenRecognition [80] 在这一步显示出潜力，但它是在移动端截图上训练的，而且是闭源的，无法在网站上使用或进行测试。

[3]https://cloud.google.com/vision/docs/ocr

(2) **Web-UI Element Detection:** Localize UI elements, extract their bounding boxes, and identify their types from the screenshot.

(3) ***ElementMap* Generation:** Merge the results of the above steps into a tabular representation of the website to generate an *ElementMap*.

4.1 Text Extraction

The Text Extraction step starts by performing Optical Character Recognition (OCR) on the website screenshot. We employ the Google Vision OCR API [3] as it has high accuracy in extracting text from images of varying scales and resolutions. The API returns blocks of detected text. We concatenate these blocks based on proximity to form coherent text blocks. These blocks are a list of bounding boxes with their respective text content. Then, we retrieve the font size, font color, and background color of each bounding box, as illustrated in Figure 3. We calculate font size by subtracting the bottom y-coordinate from the top y-coordinate of the bounding box. To determine color information, we utilize the *extcolors* [14] package, extracting the most prominent color (background) and the second most prominent color (font).

This approach addresses a key limitation identified by Soe et al. [64] in the previous ML methods: the lack of representation of the UI richness that a user perceives, such as text placement and contrast between the font and background colors. Our approach captures these detailed text features, along with the relative location of text elements on the website, providing a comprehensive representation of the textual content experienced by users.

4.2 Web-UI Element Detection

The Web-UI Element Detection step uses the same screenshot to identify and localize these 7 *Web-UI Elements*: buttons, checkboxes (☑, ■), radio buttons (●, ○), and toggle switches (◐, ◑). This step provides context to the extracted text and enables a more comprehensive understanding of the webpage's structure. For instance, distinguishing between a clickable button and a static text block can be significant, as a seemingly simple text might be a deceptive call to action when recognized as a button. Similarly, identified checkbox states (checked or unchecked) can reveal pre-selected options that users might overlook.

In the following, we describe how we survey existing Web-UI detection methods and the underlying datasets. As we find these methods and datasets to not be appropriate for the deceptive patterns detection task, we describe how we train a real-time *Web-UI Element Detector* using `YOLOv10` on a custom dataset.

Limitations of Existing Web-UI Detection Methods. While Web-UI Element Detection is a well-studied problem, the existing approaches face several limitations in our deceptive pattern detection task. Detectors like *Ominparser 2* [40], *Ferret UI 2* [38], *UEID* [76], and *Element Detector* [75] do not distinguish between checked and unchecked states. Detectors like *UISketch* [59] report low performance on real-world websites. *ScreenRecognition* [80] shows promise for this step but was trained on mobile screenshots and is closed-source, preventing its use or testing on websites.

[3]https://cloud.google.com/vision/docs/ocr

现有 Web-UI 数据集的局限性。为了提高检测器的性能，我们尝试修改用于训练它们的现有数据集。Omniparser 2 使用了手工注释的热门网站数据集，但该数据集未公开 [40]。UEID [76] 是在 RICO 数据集 [39], 上训练的，该数据集包含手工注释的移动应用 UI。我们无法修改该数据集来训练网页检测器。Ferret UI 2 [38] 和 Element Detector [75] 使用 WebUI [75], , 这是一个拥有 40 万个网站的流行框架，并且公开可用，但从无障碍树自动计算的标签 [4] 存在噪声，并且在特定网站上缺失许多元素 [74, 75]。UISketch 包含手绘图像，用于根据草图识别 Web-UI 元素，并且在真实网站上泛化能力较差 [59]。

4.2.1 数据集策划。我们开发了一种新方法来创建一个多样且具有代表性的数据集，以解决缺乏适合训练我们 Web-UI 元素检测器的 Web-UI 元素数据集的问题，如图 4 所示。我们不是依赖人工抓取和标注 [40, 59, 77, 80], , 而是利用人工智能驱动的工具生成 ）。一个反映当前网页环境的合成数据集。我们的方法可以精确控制元素的位置和标注，从而使数据集生成具有可扩展性。

我们使用 GPT-4 [3] 为多样化网站生成了 2.5K 个创意。我们将这些创意传递给 v05（请参阅我们的扩展报告 [46] 获取示例），这是一款基于 AI 的网站生成器，以生成每个创意对应的三个网站。v0 使用 shadcn 6，这是一款可定制的 UI 库，它允许我们获取每个 UI 元素的边界框和状态（选中/未选中）。研究团队的三名成员手动验证了 v0 生成的 7.5K 个网站。由于编译器能够检测网站中的错误，这个过程相对较快，失败率低于 2%。这些错误主要是拼写错误和缺失的库，由研究人员手动修复。然后，我们使用来自 6 个流行 UI 库（如 Material UI 和 Bootstrap[7]）的随机组件渲染每个 v0 生成的网站（7.5K）。这一过程生成了 62K 张截图，其中每张截图都有每个 UI 元素的边界框和标签。请注意，我们为 UI 元素定义了 7 个标签。我们将此数据集称为 t

4.2.2 训练 Web-UI 元素检测器。我们使用 Web-UI 元素数据集来训练一个 YOLOv10 模型的集成模型。具体来说，我们将数据集随机划分为一个训练集，占数据的 85%，以及一个验证集，占剩余的 15%。我们在第 4.4.2 节中展示了该训练集成模型在真实网站上的性能。

为什么选择 YOLOv10? 我们采用 YOLOv10 模型（You Only Look Once（YOLO）架构 [71], ），一种实时对象检测器，用于从截图中识别 UI 元素。我们选择 YOLOv10 而不是卷积神经网络（CNN） [16, 31, 42]和类似 Molmo 的 VLLM [23]。基于 CNN 的检测器，如 Faster R-CNN [54], , 虽然能提供高精度的预测，但需要大量的计算资源和时间 [53, 54]。尽管 VLLM 在理解图像上下文方面具有强大能力，但也表现出显著的

[4]辅助功能树是使用开发者可选择添加到网站的aria标签生成的。 [5] https://v0.dev/ [6]https://ui.shadcn.com/ [7]https://mui.com/material-ui/, https://getbootstrap.com/

Limitations of Existing Web-UI Datasets. To improve the detector performance, we explored modifying existing datasets used to train them. *Omniparser 2* uses a manually annotated dataset of popular websites, but the dataset is not public [40]. *UEID* [76] was trained on the *RICO Dataset* [39], which contains manually annotated mobile app UIs. We could not modify this dataset to train a detector for webpages. *Ferret UI 2* [38] and *Element Detector* [75] use *WebUI* [75], a popular framework with 400k websites which is publicly available, but the automatically computed labels from accessibility trees[4] are noisy and miss a lot of elements on a given website [74, 75]. *UISketch* contains hand-drawn images to identify *Web-UI Elements* based on their sketches and does not generalize well to real-world websites [59].

4.2.1 Dataset Curation. We develop a novel approach to create a diverse and representative dataset to address the lack of suitable *Web-UI Element Dataset* for training our *Web-UI Element Detector*, as shown in Figure 4. Instead of relying on manual scraping and labeling [40, 59, 77, 80], we leverage AI-powered tools to generate a synthetic dataset that reflects the current web landscape. Our approach allows for precise control over element positioning and labeling, which allows us to scale the dataset generation.

We generated 2.5K ideas for diverse websites using `GPT-4` [3]. We passed these ideas to `v0`[5] (refer to our extended report [46] for examples), an AI-based website generator, to generate three websites per idea. `v0` uses `shadcn` [6], a customizable UI-Library, which allows us to get bounding boxes and the state (checked/unchecked) of every UI element. Three members of the research team manually verified the 7.5K websites generated by `v0`. With a failure rate of less than 2%, this process was relatively fast, as errors in websites are detected by the compiler. These errors consisted mainly of typos and missing libraries that the researchers manually fixed. We then rendered each `v0` generated website (7.5K) using randomized components from 6 popular UI-libraries, like Material UI and Bootstrap[7]. This process resulted in 62K screenshots, where each screenshot had a bounding box and a label for each UI element. Recall that we have 7 labels for the UI elements. We refer to this dataset as the *Web-UI Elements Dataset*.

4.2.2 Training Web-UI Element Detector. We use the *Web-UI Elements Dataset* to train an ensemble of `YOLOv10` models. In particular, we randomly divide the dataset into a training set consisting of 85% of the images and a validation set consisting of the remaining 15%. We present the performance of the trained ensemble in Section 4.4.2 on real-world websites.

Why YOLOv10? We adopt the `YOLOv10` model (You Only Look Once (YOLO) architecture [71], a real-time object detector for recognizing UI elements from a screenshot. We chose `YOLOv10` over Convolution Neural Networks (CNNs) [16, 31, 42] and VLLMs like `Molmo` [23]. CNN-based detectors, such as *Faster R-CNN* [54], provide predictions with high accuracy, but require considerable computational power and time [53, 54]. VLLMs, despite their strong capabilities in understanding image context, demonstrate significant

[4]Accessibility trees are generated using aria labels which developers have to optionally add to a website.
[5]https://v0.dev/
[6]https://ui.shadcn.com/
[7]https://mui.com/material-ui/, https://getbootstrap.com/

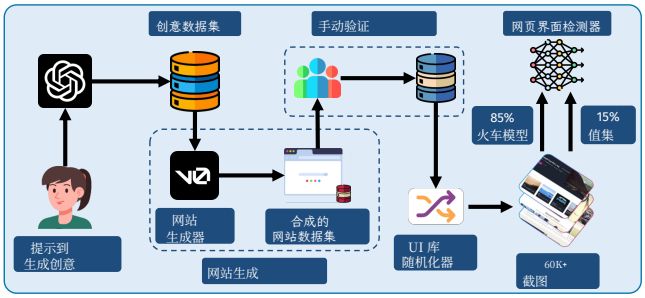


图 4：用于训练 YOLOv10 的 Web-UI 元素数据集生成流程。我们使用 GPT-4 生成了 2.5K 个创意（创意数据集），然后通过 v0 处理生成了 7.5K 个网站（合成网站数据集）。在手动检查这些网站的渲染错误并随机化其 UI 库后，我们捕获了超过 60K 张截图用于训练我们的 YOLOv10 模型。

在图像分类任务中的局限性 [81],，特别是对象检测任务 [79]。在检测 Web-UI 元素时评估 Molmo [23] 的结果不佳，如表 1 所示。YOLO 模型相对轻量，约 40MB，允许各种部署选项，而无需大量计算资源。

YOLOv10的集成。在训练过程中，我们观察到YOLOv10模型无法准确区分7个标签（表1第一列）。我们认为原因在于这些标签在视觉上相似，例如一个被选中的开关和一个被选中的单选按钮。因此，我们训练了三个YOLOv10模型，每个模型专注于区分2-3个不同的元素。第一个模型标记按钮和●；第二个模型标记 ⬬、☑和 ■；第三个模型标记 ⬭ 和 ◯。我们发现，每个模型的性能都比试图一次处理所有类别的模型好得多。文献中也曾对基于YOLO的目标检测得出过同样的观察结果 [44, 50, 70]。我们通过对三个模型检测出的UI元素进行简单的并集操作来结合它们的输出。在出现重叠的情况下，我们选择置信度更高的标签。我们将这种检测方法称为YOLOv10集成。

4.3 元素映射生成

ElementMap 生成步骤将 Web-UI 元素检测器中的 Web-UI 元素与文本提取步骤中的文本块合并。具体来说，它会遍历每一个检测到的 Web-UI 元素，并根据元素类型应用空间启发式方法，以找到最可能与该元素对应的文本块。例如，按钮是基于重叠匹配的，而复选框和单选按钮则与附近的文本配对。然后，最匹配的文本块会被重新标注为元素类型。这种标注生成了一个 ElementMap，其中每一行包含元素标签、文本、边界框坐标、字体大小、背景颜色和字体颜色。语言模块使用 ElementMap 来检测页面上的欺骗性模式。

4.4 视觉模块评估

我们创建了一个数据集来评估 YOLOv10 集成模型在现实世界中的性能。

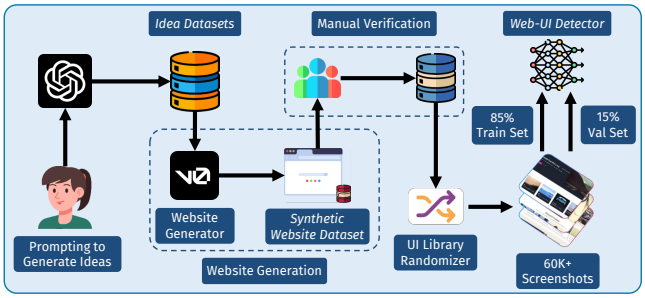


Figure 4: Pipeline of Generating Web-UI Element Dataset to train YOLOv10. We used GPT-4 to generate 2.5K ideas (*Idea Datasets*), which were then processed by `v0` to create 7.5K websites (*Synthetic Website Dataset*). After manually verifying these sites for rendering errors and randomizing their UI library, we capture over 60K screenshots to train our YOLOv10 model.

limitations in image classification tasks [81], specifically object detection tasks [79]. Evaluating `Molmo` [23] on detecting *Web-UI Elements* yielded poor results, as shown in Table 1. `YOLO` models are comparatively lightweight, around 40MB, allowing for various deployment options without requiring extensive compute resources.

Ensemble of YOLOv10. During training, we observed that `YOLOv10` models could not distinguish between the 7 labels (first column of Table 1) accurately. We attribute the reason to the labels being visually similar, such as a checked switch and a checked radio button. As such, we trained three `YOLOv10` models, each focused on distinguishing between 2-3 different elements. The first model labeled button and ●; the second labeled ⬬, ☑, and ■; and the third labeled ⬭ and ◯. We found that each model performed much better than one trying to handle all the classes at once. The same observation has been made in literature before for YOLO-based object detection [44, 50, 70]. We combined the outputs of the three models by simply performing a union over the detected UI elements. In case of overlap, we took the label with the higher confidence classification. We refer to this detection method as the `YOLOv10 Ensemble`.

4.3 *ElementMap* Generation

The *ElementMap* Generation step merges the *Web-UI Elements* from the *Web-UI Element Detector* with the text blocks from the *Text Extraction* step. In particular, it iterates over each detected *Web-UI Element* and applies spatial heuristics depending on the element type to find the most likely text block corresponding to the element. For example, buttons are matched based on overlap, while checkboxes and radio buttons are paired with nearby text. The closest matching text block is then relabeled with the element type. This labeling results in an *ElementMap*, where each row contains an element label, the text, the bounding box coordinates, the font size, the background color, and the font color. The *Language Module* uses the *ElementMap* to detect the deceptive patterns on a page.

4.4 Vision Module Evaluation

We create a dataset to evaluate the real-world performance of the `YOLOv10 ensemble`.

4.4.1 视觉数据集。我们从 Mathur 等人的欺骗模式网站数据集 [43] 中整理了一个带标签的 UI 元素数据集。我们使用 Label Studio [69] 手动标注了超过 1.5K 个网站截图。具体来说，一名作者通过在每个 UI 元素周围绘制边界框并分配其类型手动标注每个截图。类型为七种 Web-UI 元素之一：按钮、复选框 (☑, ■)、单选按钮 (◉, ○) 和切换开关 (◐, ⬭)。另一名作者独立验证了这些标注。然后，两位作者讨论并解决了标注中的冲突。我们将该数据集称为 Real-UI 数据集。

4.4.2 视觉模块评估。我们在 Real-UI 数据集上评估 YOLOv10 模型集成的准确性。我们使用 IoU 指标报告结果，该指标通过将预测边界框与真实边界框的交集面积除以它们的并集面积来衡量重叠程度。更高的 IoU 表示更好的定位准确性，我们认为当 IoU 大于 0.5 且模型置信度超过 0.3 时，检测是正确的。我们选择较低的 IoU 阈值，以考虑在合成标注数据集训练和人工标注数据集之间的分布差异。两者的边界框会有所不同。使用较高的 IoU 阈值会导致更多的假阴性，这将影响 AutoBot 流程中的后续步骤。

表 1：YOLOv10 集成模型和 Molmo [23] 在我们真实 UI 数据集上的性能

班级	精度		回忆		F1 分数		# 元素
	YOLO	Molmo	YOLO	Molmo	YOLO	Molmo	
按钮	**0.94**	0.87	**0.88**	0.41	**0.91**	0.55	5226
☑	0.85	**0.97**	**0.95**	0.47	**0.89**	0.63	113
■	**0.98**	0.92	**0.76**	0.44	**0.86**	0.60	246
◉	0.85	**0.86**	**0.93**	0.29	**0.89**	0.43	76
○	**0.86**	0.84	**0.89**	0.35	**0.87**	0.49	132
◐	**0.96**	**0.96**	**0.98**	0.42	**0.97**	0.59	52
⬭	0.91	**0.93**	**0.94**	0.47	**0.93**	0.63	34
总计	**0.91**	0.90	**0.91**	0.42	**0.91**	0.57	5879

表 1 显示了我们的 YOLOv10 集成模型和 Molmo [23] 在 7 个类别中的 F1 分数。我们观察到，该集成模型在所有 UI 元素上都优于 Molmo。请注意，对于图像输入，本地 VLLM 的少样本提示是不可能的。集成模型在未选中的单选按钮和未选中的复选框上的表现相对较低，因为它们在视觉上与其他 UI 元素非常相似。

5 语言模块

语言模块将每个存在于 ElementMap 中的元素分配一个欺骗性模式（来自第 2.1 节的分类法）。这是语言模块的输入/输出结构，如图 5 所示。

5.1 可能的解决方案

以往的研究表明，大型语言模型（LLMs） [45] 适合类似于我们任务的推理任务。这些模型可分为三类：1）大型语言模型（LMs），如 Gemini 和 GPT-4，2）小型语言模型，如 Gemma 和 Qwen，以及 3）非常小的语言模型，如 T5。这些

4.4.1 Vision Dataset. We curate a labeled dataset of UI elements from the deceptive pattern websites dataset of Mathur et al. [43]. We manually annotated over 1.5K website screenshots using Label Studio [69]. In particular, one author manually annotated each screenshot by drawing bounding boxes around each UI element and assigning it a type. The type is one the 7 *Web-UI Elements*: buttons, checkboxes (☑, ■), radio buttons (◉, ○), and toggle switches (◐, ⬭). Another author independently verified the annotations. Both authors then discussed and resolved the conflicts in annotations. We refer to this dataset as the *Real-UI Dataset*.

4.4.2 Vision Module Evaluation. We measure the accuracy of the `YOLOv10 ensemble` of models on the *Real-UI Dataset*. We report our results using the IoU metric, which measures the overlap between the predicted bounding box and the ground truth box by dividing the area of their intersection by the area of their union. A higher IoU indicates better localization accuracy, and we consider a detection to be correct if the IoU exceeds 0.5 and the model confidence exceeds 0.3. We choose a lower IoU threshold to account for the distribution shift between training on a synthetically labeled dataset and a human-annotated one. The bounding boxes from both will be different. Using a high IoU threshold would result in more false negatives, which would affect the subsequent steps in *AutoBot*'s pipeline.

Table 1: Performance of the *YOLOv10 Ensemble* and *Molmo* [23] on our *Real-UI Dataset*

Class	Precision		Recall		F1-Score		# Elements
	YOLO	Molmo	YOLO	Molmo	YOLO	Molmo	
button	**0.94**	0.87	**0.88**	0.41	**0.91**	0.55	5226
☑	0.85	**0.97**	**0.95**	0.47	**0.89**	0.63	113
■	**0.98**	0.92	**0.76**	0.44	**0.86**	0.60	246
◉	0.85	**0.86**	**0.93**	0.29	**0.89**	0.43	76
○	**0.86**	0.84	**0.89**	0.35	**0.87**	0.49	132
◐	**0.96**	**0.96**	**0.98**	0.42	**0.97**	0.59	52
⬭	0.91	**0.93**	**0.94**	0.47	**0.93**	0.63	34
Total	**0.91**	0.90	**0.91**	0.42	**0.91**	0.57	5879

Table 1 shows the F1-scores of our `YOLOv10 Ensemble` and `Molmo` [23] for each of the 7 classes. We observe that the ensemble outperforms `Molmo` for all the UI elements. Note that few-shot prompting of local VLLMs is not possible for image inputs. The ensemble exhibits a relatively lower performance on the unchecked radio button and unchecked check box, because they look visually similar to the other UI elements.

5 Language Module

The *Language Module* assigns a deceptive pattern (from the taxonomy in Section 2.1) to each element present in an *ElementMap*. This is the input/output structure of the language module as shown in Figure 5.

5.1 Possible Solutions

Prior works have shown that LLMs [45] are suitable for reasoning tasks similar to our task. These models fall into three categories: 1) large Language Models (LMs) like `Gemini` and `GPT-4`, 2) small LMs such as `Gemma` and `Qwen`, and 3) very small LMs like `T5`. These

101

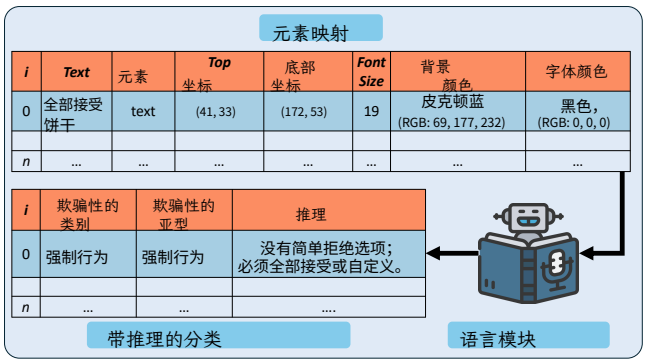


图5：我们的语言模块的输入和输出结构。输入ElementMap由网页元素的关键特征组成，输出包含一个欺骗类别、子类型和分类推理。

正如表2所示，模型具有不同的能力和权衡。

表2：语言模型对比：Gemini vs Qwen2.5 vs T5

特征	大型语言模型	小型语言模型	非常小的语言模型
大小(参数) 大 (>2		中等（1.5B）	非常小（700M）
上下文窗口	1M	128K	<1千
部署	仅限云 API	可以本地运行	可以本地运行
所需内存 不适用		~4.5 GB	~700MB
许可证	专有	开源	开源
延迟	更高	中等	非常低
Cost	高自由自由		
数据隐私	数据离开设备	数据保留在设备上	数据保留在设备上

5.1.1 大型语言模型（LLMs）：LLMs 在执行各种任务方面非常有效 [5, 21, 62]，并且能够紧密遵循用户指令 [49, 82]。然而，如表 2 所示，使用这些 LLM 的成本很高，并且存在潜在的数据隐私问题 [8]。

5.1.2 小型语言模型（SLMs）：与在执行任务时遵循复杂指令的LLMs不同，SLMs往往难以做到这一点。通过在特定任务上对SLMs进行微调，可以克服这一局限性，从而提升其性能以匹配LLMs [41]。此外，如表2所示，SLMs相比LLMs有两个主要优势：1）计算效率高，可以在各种平台上本地部署；2）缓解了与基于API的LLMs相关的任何数据隐私问题。

5.1.3 非常小型语言模型 (vSLMs)：SLMs 虽然计算效率高，但并不是在极度资源受限的平台上使用的最佳解决方案，例如在浏览器中或没有专用 GPU 的设备上。在这种环境中，我们可以利用像 Flan-T5 [18] 这样的 vSLMs。像 Flan-T5 这样的模型需要针对特定任务进行微调或从大型语言模型 (LLMs) 中蒸馏，以实现高准确性 [29]。我们在第 5.2.4 节展示了我们执行的详细蒸馏步骤。

总之，我们观察到大型语言模型，如Gemini，在检测欺骗性模式方面相当有效

[8] https://www.traceable.ai/2023-state-of-api-security

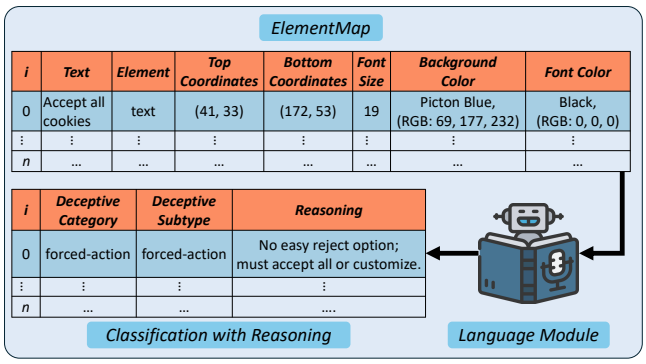


Figure 5: The input and output structure of our language module. The input ElementMap consists of key features of a web element, and the output contains a deceptive category, subtype, and reasoning of classification.

models have varying capabilities and trade-offs, as described in Table 2.

Table 2: Comparison of Language Models: `Gemini` vs `Qwen2.5` vs `T5`

Feature	Large LM	Small LM	Very Small LM
Size (parameters)	Large (>200B)	Medium (1.5B)	Very Small (700M)
Context Window	1M	128K	<1K
Deployment	Cloud API only	Can be run locally	Can be run locally
Required Memory	N/A	~4.5 GB	~700MB
License	Proprietary	Open Source	Open Source
Latency	Higher	Medium	Very low
Cost	High	Free	Free
Data Privacy	Data leaves device	Data stays on device	Data stays on device

5.1.1 Large Language Models (LLMs): LLMs are very effective at performing a wide range of tasks [5, 21, 62] and are able to closely follow user instructions [49, 82]. However, as shown in Table 2, using these LLMs is cost-prohibitive and has potential data privacy concerns[8].

5.1.2 Small Language Models (SLMs): Unlike LLMs that follow complex instructions when performing a task, SLMs often struggle to do so. This limitation of SLMs can be overcome by fine-tuning them on specific tasks, improving their performance to match that of LLMs [41]. Moreover, as shown in Table 2, SLMs have two key advantages over LLMs: 1) being compute efficient, they can be deployed locally across a wide range of platforms, and 2) they alleviate any data privacy concern associated with API based LLMs.

5.1.3 Very Small Language Models (vSLMs): SLMs, while compute-efficient, are not the best solution to use in an extremely resource-constrained platform, such as in-browser or on devices with no specialized GPU. In such environments, we can leverage vSLMs like `Flan-T5` [18]. Models like `Flan-T5` need to be finetuned or distilled from LLMs for specific tasks to achieve high accuracy [29]. We show the detailed distillation steps we performed in Section 5.2.4.

In summary, we observe that large LMs, such as `Gemini`, are considerably effective in detecting deceptive patterns from an

[8] https://www.traceable.ai/2023-state-of-api-security

102

ElementMap。然而，如表2所述，使用如此大型且封闭源的模型会带来一些挑战，例如高使用成本、显著的延迟以及由于ElementMap 被发送到外部服务而可能产生的数据隐私问题。较小的语言模型如 Qwen 和 T5 可以应对这些挑战，并且经微调后已被证明在此类特定任务上表现良好 [29, 41]。

5.2 我们的解决方案

为了结合大语言模型和小语言模型的优势，我们采用了一种蒸馏方法，在该方法中，我们使用大语言模型作为教师，小语言模型作为学生来完成我们的任务。具体来说，我们利用 Gemini 从 ElementMap 创建了一个欺骗模式分类的合成数据集。然后，我们使用该数据集对较小的学生模型进行蒸馏，即 Qwen 和 T5。因此，AutoBot 包含三个用于检测欺骗模式的语言模型，每个模型都呈现出不同的权衡，如表 2 所示。

5.2.1 提示大型语言模型（LLM）。我们利用经过验证的技术，例如连锁思维（Chain-of-Thought, CoT） [73]，、少样本提示 [13]，以及提示模型对其分类进行推理 [28, 45, 48]，以帮助模型理解任务并识别欺骗性模式。具体来说，我们的系统提示 P_{system}[9] 为 LLM 提供了检测欺骗性模式的计划，并指示 LLM 为 ElementMap 中的每个元素生成三列——欺骗类别、欺骗子类型和推理，如图 5 所示。该提示不仅使我们能够生成精确的标签及相关推理，同时也能最大程度地减少幻觉。生成带有“推理”的分类的另一个好处是，我们可以使用这些“理由”进一步训练较小的语言模型（LM）。这种逐步蒸馏（Distilling Step-by-Step）的方法已被 Hsieh 等人 [29] 证明非常有效。

在我们使用 P_{system} 对 Gemini 1.5 Pro 进行初步评估时，我们观察到该模型能够识别 ElementMap 中所有的欺骗模式，但常常会将“非欺骗”模式错误地分类为欺骗，从而导致高假阳性率和较低的精确度分数。为了解决这个问题，我们使用了 Gemini 2.0-Flash-Thinking 推理模型。推理模型在能够对任务 [41, 48] 进行推理方面表现出有希望的结果。我们提示大型语言模型重新评估被识别为欺骗的元素，并纠正错误分类 [9]。接下来，我们利用 Gemini 2.0-Flash-Thinking 模型对每个具有一个或多个被分类为欺骗元素的网站重新验证标签。这一额外的标签生成步骤显著减少了我们观察到的假阳性数量。我们注意到，我们没有将该模型作为基础模型使用，因为其可用性有限，无法大规模使用。我们展示了这种方法能够获得较高的

5.2.2 创建蒸馏数据集。我们通过抓取和分析 Tranco 列表中的 11K 个网站来创建 $\mathcal{D}$distill [51]。我们使用 langdetect 库过滤掉非英语网站 [63]，并使用 NudeNet 包过滤成人网站 [47],，最终剩下 6,626 个网站。对于这些网站，其中很大一部分是非欺骗性的或只有极少的欺骗模式。为了增加具有欺骗模式的网站数量，我们选择查看电子商务网站，

[9]请在此链接的文件中找到系统提示：https://osf.io/tha2d/?view_only=4fd2116fa2e94c99857679eddfea5937

ElementMap. However, as described in Table 2, utilizing such large and closed-source models presents challenges like high usage cost, considerable latency, and potential data-privacy concerns as the *ElementMap* is sent to an external service. Smaller LMs such as `Qwen` and `T5` address these challenges and have been shown to perform well on such specific tasks after finetuning [29, 41].

5.2 Our Solution

To combine the strengths of large and small LMs, we adopt a distillation approach where we use large LMs as teachers and small LMs as students to complete our task. Specifically, we create a synthetic dataset of deceptive pattern classification from *ElementMap* using `Gemini`. We then use this dataset to distill smaller student models, i.e., `Qwen` and `T5`. As such, *AutoBot* comprises three language models to detect deceptive patterns, each presenting different trade-offs as described in Table 2.

5.2.1 Prompting the LLM. We utilize proven techniques like Chain-of-Thought (CoT) [73], few-shot prompting [13], and prompting the model to reason about its classification [28, 45, 48] to help the model understand the task and identify deceptive patterns. Specifically, our system prompt, P_{system}[9], provides the LLM with a plan on how to detect deceptive patterns and instructs the LLM to generate three columns — *Deceptive Category*, *Deceptive Subtype*, and *Reasoning* — for each element in the *ElementMap*, as shown in Figure 5. This prompt allows us to not only generate precise labels and the associated reasoning but also minimize hallucinations. An added benefit of generating the classification with 'Reasoning' is that we can use these 'rationales' to further train smaller LMs. This *Distilling Step-by-Step* methodology has been shown to be very effective by Hsieh et al. [29].

During our initial evaluation of `Gemini 1.5 Pro`, using P_{system}, we observed that the model could identify all deceptive patterns in the *ElementMap*, but would often mis-classify 'non-deceptive' patterns as deceptive, resulting in a high false positive rate and a low precision score. To address this problem, we utilized the `Gemini 2.0-Flash-Thinking` reasoning model. Reasoning models have shown promising results in being able to reason about a task [41, 48]. We prompted the LLM to re-evaluate the elements identified as deceptive and correct misclassifications[9]. Next, we utilized the Gemini 2.0-Flash-Thinking model to re-verify the labels on every website with one or more elements classified as deceptive. This additional step in generating the labels, significantly reduced the number of false positives we observed. We note that we do not use this model as our base model because of its limited availability, making it infeasible to be used at scale. We show that this approach results in high precision and recall in Section 5.3.

5.2.2 Creating Distillation Dataset. We create $\mathcal{D}_{\text{distill}}$ by scraping and analyzing 11K websites from the Tranco list [51]. We filter out non-English websites using the `langdetect` library [63] and adult websites using the `NudeNet` package [47], leaving us with 6,626 websites. For these websites, a significant portion was non-deceptive or had very few deceptive patterns. To increase websites with deceptive patterns, we opted to look at e-commerce websites,

[9]Please find the system prompt under *Files* here: https://osf.io/tha2d/?view_only=4fd2116fa2e94c99857679eddfea5937

因为这些网站往往具有欺骗性模式 [43]。因此，我们分析了来自Shopify合作伙伴目录的另外4,492个特色网站[10]。

总体而言，我们的数据集中共有 11,118 个网站。我们在这些网站上运行 AutoBot，使用 Gemini 1.5 Pro 和 Gemini 2.0 Flash-Thinking 模型。为了减少随机性并获得确定性的输出，我们限制了 $temperature = 0$ 和 $top_p = 0.1$ [55]。我们将 AutoBot 在 $\mathcal{D}$ distill 中生成的最终分类和推理纳入其中。样本分布（样本定义为 ElementMap 的单行）如表 3 所示。

表 3：$\mathcal{D}_{\text{distill}}$ 中样本的分布。

类别	亚型	# 样本
不欺骗	不适用	160934
强制行动	强制行动	3403
界面干扰	轻推	1335
	假稀缺 / 假紧迫	933
	确认羞辱	428
阻塞	视觉干扰	689
	预选	234
偷偷地移动	巧妙措辞	904
	隐性成本	233
	隐藏订阅	3495
	伪装广告	5980

5.2.3 蒸馏小型语言模型（Qwen2.5-1.5B）。Deep-ScaleR [41] 已经表明，针对特定任务微调的小型语言模型（SLMs）能够在相同任务上模拟大型模型的表现。利用这一见解，我们对 Qwen2.5-1.5B 模型进行蒸馏，使其能够模仿 Gemini 在检测欺骗性模式方面的表现。我们使用 $\mathcal{D}$ 蒸馏数据集对 Qwen2.5-1.5B 模型进行完整微调。如表 3 所示，蒸馏数据集中样本的分布高度不平衡，其中超过 92% 的样本为“非欺骗性”。在如此不平衡的数据集上训练可能会引入对多数类的偏向 [34]。为了减轻这种偏差，我们对“非欺骗性”类别进行了随机下采样，这已被证明可以提高性能 [24], ，从而实现约 55% “非欺骗性”样本的更平衡分布。我们蒸馏后的 Qwen2.5-1.5B 版本在 4 块 Nvidia-A6000 GPU 上训练了 2 个周期，共处理 3470 万个 token。我们预

5.2.4 蒸馏超小型语言模型（T5）：为了将大语言模型（LLM）的知识蒸馏到像 T5 这样的超小型语言模型中，我们使用并改进了 Hsieh 等人 [29] 提出的范式。我们将 $\mathcal{D}$distill 划分为 90% 的训练集和 10% 的验证集，并按样本提取的站点进行分组。这确保了来自同一站点的样本不会同时出现在训练集和测试集中。我们正式将训练数据集定义为 D_{train}，如下所示：

$$d_i \in D_{\text{train}} \subset \mathcal{D}_{\text{distill}} \tag{1}$$

$$d_i = (x_i, y_i, z_i, r_i) \tag{2}$$

其中 D_{train} 是第5.2.3节中平衡 $\mathcal{D}$ 提取的一个子集。这里，x_i 代表用于分类的输入，y_i 代表欺骗性

[10]https://www.shopify.com/partners/directory

as these websites tend to have deceptive patterns [43]. As such, we analyze an additional 4,492 featured websites from *Shopify Partners Directory*[10].

Overall, we have 11,118 websites in our dataset. We run *AutoBot* on these websites, using the `Gemini 1.5 Pro` and `Gemini 2.0 Flash-Thinking` models. To reduce randomness and have deterministic outputs, we limit the $temperature = 0$ and $top_p = 0.1$ [55]. We incorporate the final classification and reasoning produced by *AutoBot* in $\mathcal{D}_{\text{distill}}$. The distribution of samples (a sample is defined as a single row of the *ElementMap*) is shown in Table 3.

Table 3: Distribution of Samples in $\mathcal{D}_{\text{distill}}$.

Category	Subtype	# Samples
Non Deceptive	*Not Applicable*	160934
Forced Action	*Forced Action*	3403
Interface Interference	*Nudge*	1335
	Fake Scarcity / Fake Urgency	933
	Confirmshaming	428
Obstruction	*Visual Interference*	689
	Pre-Selection	234
Sneaking	*Trick Wording*	904
	Hidden Costs	233
	Hidden Subscription	3495
	Disguised Ads	5980

5.2.3 Distilling Small Language Models (Qwen2.5-1.5B). Deep-ScaleR [41] has shown that small language models (SLMs) fine-tuned for specific tasks are able to mirror the performance of larger models on those same tasks. Using this insight, we distill a `Qwen2.5-1.5B` model to be able to mimic `Gemini`'s performance at detecting deceptive patterns. We use the $\mathcal{D}_{\text{distill}}$ dataset to perform full-finetuning of the `Qwen2.5-1.5B` model. As shown in Table 3, the distribution of samples in the distillation dataset is highly imbalanced, with over 92% of the samples being ‘non-deceptive’. Training on such an imbalanced dataset may introduce bias towards the majority class [34]. To mitigate such bias, we perform Random Under Sampling of the ‘non-deceptive’ class, which has shown to improve performance [24], to achieve a more balanced distribution of about 55% ‘non-deceptive’ samples. Our distilled version of `Qwen2.5-1.5B` was trained on 34.7M tokens on 4 Nvidia-A6000 GPUs for 2 epochs. We present the performance of the trained SLM in Section 5.3.

5.2.4 Distilling Very Small Language Models (T5): To distill the knowledge from LLMs into very small language models like `T5`, we use and improve the paradigm introduced by Hsieh et al. [29]. We split $\mathcal{D}_{\text{distill}}$ into 90% training and a 10% validation set grouped by the sites from which the samples were extracted. This ensures that samples from the same site are not present in training and testing sets. We formally define the training dataset, D_{train}, as:

$$d_i \in D_{\text{train}} \subset \mathcal{D}_{\text{distill}} \tag{1}$$

$$d_i = (x_i, y_i, z_i, r_i) \tag{2}$$

where D_{train} is a subset of the balanced $\mathcal{D}_{\text{distill}}$ from Section 5.2.3. Here, x_i represents the input to classify, y_i represents the deceptive

[10]https://www.shopify.com/partners/directory

103

设计类别， z_i 表示欺骗性设计子类型， r_i 表示该类别和子类型的相关推理。由于T5的上下文窗口明显小于SLMs和LLMs，对于每个需要分类的网页元素 x_i，我们以滑动窗口的方式提供其相邻的网页元素：$x_{i-1\to i-n}$ 到 $x_{i+1\to i+n}$。对于蒸馏任务，我们使用了 $n = 4$。

基线方法。基于这一初始方法，我们在一个多任务问题上训练了一个 T5 模型 f：预测欺骗性设计的类别，将子类型作为标签，并将推理作为理由。我们使用任务特定的前缀 [classify] 来生成标签，以及[reason] 来生成推理。

我们将模型和损失函数定义为：

$$f(x,t) = \begin{cases} (\hat{y}_i \oplus \hat{z}_i), & \text{if } t = \texttt{[category]} \\ \hat{r}_i, & \text{if } t = \texttt{[reason]} \end{cases} \tag{3}$$

$$\mathcal{L} = \mathcal{L}_{\text{label}} + \alpha \mathcal{L}_{\text{reason}} \tag{4}$$

这里， $\mathcal{L}$label 和 $\mathcal{L}$reason 分别是标签预测损失和原因生成损失，其定义如下：

$$\mathcal{L}_{\text{label}} = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i, t_{\text{category}}), y_i \oplus z_i) \tag{5}$$

$$\mathcal{L}_{\text{reason}} = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i, t_{\text{reason}}), r_i), \tag{6}$$

其中 ℓ 是预测标记和目标标记之间的交叉熵损失， $\oplus$ 是一个字符串拼接函数。

每个要分类的网页元素 x_i，都会与其相邻的网页元素 $x_{i-1\to 0}$ 到 $x_{i+1\to N}$ 一起提供。模型的性能通过其 [类别] 输出与真实数据的完全匹配来衡量。下面展示了模型的示例输入和预期输出。

输入示例

输入： [类别]: 第14行，偏好设置，已选中复选框-盒子，...</s>第10行，"MAGIC 我们使用Cookies来个性化 ...</s>第11行，即将推出，文本,...</s>

输出：阻塞，预选

输入： [原因]: 第14行，偏好设置，已勾选盒子，...</s>第10行，"MAGIC 我们使用Cookies来个性化 ...</s>第11行 ,即将到来 ,text,...</s>

输出：*Cookie* 横幅选项预先选中以提示用户允许额外的饼干。

在对T5进行 $\mathcal{D}$distill数据集训练2个周期后，我们观察到训练准确率达到 ~48%。这里，准确率是指对类别和子类型的正确预测。

我们的方法：为了克服基线方法的低准确性，我们将标注任务拆分为两个独立的任务：类别和子类型，同时引入额外的“任务前缀” [子类型] 用于

design category, z_i represents the deceptive design subtype, and r_i represents the associated reasoning for the category and subtype. Since, the context window of T5 is significantly smaller than that of SLMs and LLMs, for each web element that is to be classified, x_i, we provide its neighboring web elements, in a sliding window fashion: $x_{i-1\to i-n}$ to $x_{i+1\to i+n}$. For the distillation task, we used $n = 4$.

Baseline Approach. Based on this initial methodology, we train a T5 model, f, on a multi-task problem: predict the deceptive design category, subtype as the label, and reasoning as the rationale. We use the task-specific prefix [classify] to generate the label and [reason] for the reasoning.

We define the model and loss functions as:

$$f(x,t) = \begin{cases} (\hat{y}_i \oplus \hat{z}_i), & \text{if } t = \texttt{[category]} \\ \hat{r}_i, & \text{if } t = \texttt{[reason]} \end{cases} \tag{3}$$

$$\mathcal{L} = \mathcal{L}_{\text{label}} + \alpha \mathcal{L}_{\text{reason}} \tag{4}$$

Here, $\mathcal{L}_{\text{label}}$ and $\mathcal{L}_{\text{reason}}$ are the label prediction loss and reason generation loss, respectively, and are defined as:

$$\mathcal{L}_{\text{label}} = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i, t_{\text{category}}), y_i \oplus z_i) \tag{5}$$

$$\mathcal{L}_{\text{reason}} = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i, t_{\text{reason}}), r_i), \tag{6}$$

where ℓ is the cross entropy loss between the predicted and target tokens, and $\oplus$ is a string concatenation function.

Each web element that is to be classified, x_i, is provided alongside its neighboring web elements, $x_{i-1\to 0}$ to $x_{i+1\to N}$. The model's performance is measured as the exact match of its [category] outputs with the ground truth data. An example of the model's sample input and expected output is shown below.

Input Sample

Input: [category]: Line 14,Preferences,checked checkbox,...</s>Line 10,"MAGIC We use cookies to personalise ...</s>Line 11,COMING SOON ,text,...</s>

Output: *obstruction,pre-selection*

Input: [reason]: Line 14,Preferences,checked checkbox,...</s>Line 10,"MAGIC We use cookies to personalise ...</s>Line 11,COMING SOON ,text,...</s>

Output: *Cookie banner option is pre-selected to indicate users to allow extra cookies.*

After training T5 on the $\mathcal{D}_{\text{distill}}$ dataset for 2 epochs , we observed the training accuracy to saturate to ~48%. Here, accuracy refers to the correct prediction of both category and sub-types.

Our Approach: To overcome the low accuracy in the baseline approach, we split the labeling task into two separate tasks: category and subtype, introducing an additional "task prefix" [subtype] for

新的任务。我们重新定义模型和损失函数为：

$$f(x,t) = \begin{cases} \hat{y}_i, & \text{if } t = \texttt{[category]} \\ \hat{z}_i, & \text{if } t = \texttt{[subtype]} \\ \hat{r}_i, & \text{if } t = \texttt{[reason]} \end{cases} \tag{7}$$

$$\mathcal{L} = \alpha(\mathcal{L}_{\text{category}} + \mathcal{L}_{\text{subtype}}) + (1-\alpha)\mathcal{L}_{\text{reason}}, \tag{8}$$

其中 α 是一个调节因子。

通过将标签分为类别和子类型，除了原因任务之外，模型可以学习类别与子类型之间的关系以及它们如何与推理相关。下面显示了新输入和预期输出的示例。

输入示例

输入： [类别]: 第14行，偏好设置，已勾选 检查box,...</s>第10行 ,魔法 我们使用 cookies 来个性化 ...</s>第11行 ,即将到来 ,text,...</s>

输出：阻碍

输入： [子类型]: Line 14,偏好,已选 检查box,...</s>第10行 ,魔法 我们使用 cookies 来个性化 ...</s>第11行，即将推出 ,文本,...</秒>

输出：预选择

输入： [原因]: Line 14，偏好，已选 检查盒子，...</s>第10行，"MAGIC 我们使用Cookies来个性化 ...</s>第11行，即将推出，文本,...</s>

输出：*Cookie* 横幅选项预先选中以提示用户允许额外的饼干。

在使用相同的真实数据集和相同的训练轮数，基于新的损失函数和任务对 T5 模型进行训练之后，我们观察到检测欺骗模式的测试准确率饱和到 ~95%。我们在以下部分展示这些结果。

5.3 语言模块评估

我们创建了一个数据集来评估不同模型在语言模块中的实际表现。

5.3.1 语言数据集。为了评估我们管道的语言模块，我们整理了一个数据集，称为 LangEval 数据集。具体而言，我们从 D3 数据集（特别是 Mathur 等人的部分，如第 6.1 节所述）中随机选择 200 个网站。接下来，对于这 200 个网站，其中一位作者手动标注了 ElementMap 中的 UI 元素分类，以提供真实的 UI 标签。因此，LangEval 数据集包含每个网站的手动标注 ElementMap，并与手动标注的欺骗模式相关联。

5.3.2 评估语言模型。我们在 LangEval 数据集上评估本节讨论的各种语言模型。性能如表 4 所示。我们以两种方式报告语言模型的性能：

the new task. We redefine the model and loss function to:

$$f(x,t) = \begin{cases} \hat{y}_i, & \text{if } t = \texttt{[category]} \\ \hat{z}_i, & \text{if } t = \texttt{[subtype]} \\ \hat{r}_i, & \text{if } t = \texttt{[reason]} \end{cases} \tag{7}$$

$$\mathcal{L} = \alpha(\mathcal{L}_{\text{category}} + \mathcal{L}_{\text{subtype}}) + (1-\alpha)\mathcal{L}_{\text{reason}}, \tag{8}$$

where α is a tuning factor.

By separating `label` into `category` and `subtype` in addition to the `reason` tasks, the model can learn the relation between the category and the subtype and how they both relate to the reasoning. A sample of the new inputs and expected outputs is shown below.

Input Sample

Input: [category]: Line 14,Preferences,checked checkbox,...</s>Line 10,"MAGIC We use cookies to personalise ...</s>Line 11,COMING SOON ,text,...</s>

Output: *obstruction*

Input: [subtype]: Line 14,Preferences,checked checkbox,...</s>Line 10,"MAGIC We use cookies to personalise ...</s>Line 11,COMING SOON ,text,...</s>

Output: *pre-selection*

Input: [reason]: Line 14,Preferences,checked checkbox,...</s>Line 10,"MAGIC We use cookies to personalise ...</s>Line 11,COMING SOON ,text,...</s>

Output: *Cookie banner option is pre-selected to indicate users to allow extra cookies.*

After training the T5 model on the new loss function and tasks using the same ground truth dataset and the same number of epochs, we observe the test accuracy of detecting deceptive patterns saturate to ~95%. We present these results in the following section.

5.3 Language Module Evaluation

We create a dataset to evaluate the real-world performance of the different models in the language module.

5.3.1 Language Dataset. To evaluate the *Language Module* of our pipeline, we curate a dataset, referred to as *LangEval* dataset. In particular, we randomly choose 200 websites from the *D3 Dataset* (in particular the Mathur et al. portion) described in Section 6.1. Next, for these 200 websites, one of the authors manually annotated the UI element classifications in the *ElementMap* to provide ground truth UI labels. Thus, the *LangEval* dataset contains the manually labeled *ElementMap* of each website associated with the manually labeled deceptive patterns.

5.3.2 Evaluating Language Models. We evaluate the various language models discussed in this section on the *LangEval* dataset. The performance is shown in Table 4. We report the performance of the language models in two ways:

104

1. 二元分类：该指标评估模型检测用户界面元素是否具有欺骗性的能力，而不考虑具体的分类。我们使用此指标，因为它在考虑欺骗模式分类固有主观性的同时提供性能度量——一个被归类为“陷阱措辞”的元素也可能被归类为“隐藏成本”。此分类信息显示在每个评估表的底部，标签为：“欺骗性”和“非欺骗性”。

2. 按类别分类：这是详细的分类结果，显示了各类别及子类型之间的真实数据与模型生成结果的对比。

表 4：不同模型在 LangEval 的 LanguageModule 上的表现。表格分为三个部分，分别展示类别、子类型和二元级别的表现

	图案类型	精度			回忆			F1 分数		
		双子座	Qwen	T5	双子座	Qwen	T5	双子座	Qwen	T5
Category	**不欺骗**	**1.00**	0.98	**1.00**	**0.99**	**0.99**	0.93	**0.99**	0.98	0.96
	强制行动	**0.89**	0.85	0.84	0.79	**0.83**	**0.86**	0.84	0.84	**0.85**
	界面干扰	**0.85**	0.79	0.51	**0.85**	0.55	0.69	**0.85**	0.65	0.59
	阻塞	**0.53**	0.49	0.17	0.91	0.95	**1.00**	**0.67**	0.64	0.29
	偷偷地移动	**0.87**	0.73	0.50	**0.96**	0.71	0.84	**0.91**	0.72	0.63
Subtype	不适用	**1.00**	0.98	0.99	**0.99**	**0.99**	0.89	**0.99**	0.98	0.94
	确认羞辱	0.80	**0.90**	0.86	0.80	0.82	**1.00**	**0.80**	0.86	0.92
	伪装广告	**0.76**	0.58	0.33	**0.96**	0.58	0.73	**0.85**	0.58	0.46
	虚假稀缺/虚假的紧迫感	**0.96**	0.91	0.61	**0.93**	0.63	0.76	**0.95**	0.75	0.68
	强制行动	**0.89**	0.85	0.78	0.79	0.83	**0.88**	**0.84**	**0.84**	0.82
	隐性成本	**0.60**	-	0.05	**0.60**	-	0.20	**0.60**	-	0.08
	隐藏订阅	**0.90**	0.78	0.56	**0.95**	0.72	0.77	**0.92**	0.75	0.64
	轻推	**0.74**	0.57	0.17	**0.80**	0.37	0.68	**0.77**	0.45	0.28
	预选	**0.67**	0.65	0.33	**1.00**	**1.00**	**1.00**	**0.80**	0.79	0.50
	巧妙措辞	**0.86**	0.61	0.34	**0.80**	0.61	0.47	**0.83**	0.61	0.39
	视觉的干扰	**0.50**	0.36	0.06	0.83	0.80	**1.00**	**0.62**	0.50	0.11
	欺骗性的	**0.89**	0.84	0.65	**0.95**	0.80	**0.95**	**0.92**	0.82	0.77
	不欺骗	**1.00**	0.98	0.99	**0.99**	**0.99**	0.95	**0.99**	0.98	0.97

这些结果突显了不同语言模型之间的权衡。正如预期的那样，Gemini 展现出最高的性能，在大多数欺骗模式上几乎达到了完美的精确度和召回率。它在两个模式子类型上表现不佳：隐藏成本和视觉干扰。第二名是 Qwen 模型，它在更多子类型上表现不佳。蒸馏的 T5 模型总体上表现可接受，但在以下模式类别中表现欠佳：界面干扰和阻碍。

6 端到端评估

我们在一个真实世界的数据集上对 AutoBot 进行了端到端评估，该数据集由 1.1K 个网站组成，由两位作者手工标注（在第 6.1 节中描述）。我们使用 LanguageModule 中的所有三个大型语言模型进行评估（见第 5.1 节）。在此次端到端评估中，我们的目标是衡量整个框架（包括视觉模型和语言模型）在检测网站欺骗模式方面的准确性。

6.1 数据集

为了创建用于端到端评估的数据集，我们从 Mathur 等人的欺骗模式数据集 [43] 中爬取网站。我们使用这个数据集，因为它是最新且最全面的具有欺骗模式的网站数据集。我们从该数据集中筛选出非英文和离线网站，剩下 555 个网站。

1. *Binary Classification:* This metric assesses models’ ability to detect whether a UI element *is deceptive*, regardless of the specific categorization. We use this metric as it provides a performance measure while considering the inherent subjectivity of deceptive pattern classification – an element classified as ‘trick-wording’ could also be classified as ‘hidden-cost’. This classification is provided at the bottom of each evaluation table, with labels: “Deceptive” and “Non-Deceptive”.
2. *Class-wise Classification:* This is the *detailed classification* result, across categories and subtypes, between the ground truth data and the model-generated result.

Table 4: Performance of different models in *LanguageModule* on *LangEval*. The table has three sections, each showing performance at the category, subtype, and binary levels

	Pattern Type	Precision			Recall			F1-Score		
		Gemini	Qwen	T5	Gemini	Qwen	T5	Gemini	Qwen	T5
Category	**Non Deceptive**	**1.00**	0.98	**1.00**	**0.99**	**0.99**	0.93	**0.99**	0.98	0.96
	Forced Action	**0.89**	0.85	0.84	0.79	**0.83**	**0.86**	0.84	0.84	**0.85**
	Interface Interference	**0.85**	0.79	0.51	**0.85**	0.55	0.69	**0.85**	0.65	0.59
	Obstruction	**0.53**	0.49	0.17	0.91	0.95	**1.00**	**0.67**	0.64	0.29
	Sneaking	**0.87**	0.73	0.50	**0.96**	0.71	0.84	**0.91**	0.72	0.63
Subtype	*Not Applicable*	**1.00**	0.98	0.99	**0.99**	**0.99**	0.89	**0.99**	0.98	0.94
	Confirmshaming	0.80	**0.90**	0.86	0.80	0.82	**1.00**	**0.80**	0.86	0.92
	Disguised Ads	**0.76**	0.58	0.33	**0.96**	0.58	0.73	**0.85**	0.58	0.46
	Fake Scarcity / Fake Urgency	**0.96**	0.91	0.61	**0.93**	0.63	0.76	**0.95**	0.75	0.68
	Forced Action	**0.89**	0.85	0.78	0.79	0.83	**0.88**	**0.84**	**0.84**	0.82
	Hidden Costs	**0.60**	-	0.05	**0.60**	-	0.20	**0.60**	-	0.08
	Hidden Subscription	**0.90**	0.78	0.56	**0.95**	0.72	0.77	**0.92**	0.75	0.64
	Nudge	**0.74**	0.57	0.17	**0.80**	0.37	0.68	**0.77**	0.45	0.28
	Pre-Selection	**0.67**	0.65	0.33	**1.00**	**1.00**	**1.00**	**0.80**	0.79	0.50
	Trick Wording	**0.86**	0.61	0.34	**0.80**	0.61	0.47	**0.83**	0.61	0.39
	Visual Interference	**0.50**	0.36	0.06	0.83	0.80	**1.00**	**0.62**	0.50	0.11
	Deceptive	**0.89**	0.84	0.65	**0.95**	0.80	**0.95**	**0.92**	0.82	0.77
	Not Deceptive	**1.00**	0.98	0.99	**0.99**	**0.99**	0.95	**0.99**	0.98	0.97

These results highlight the trade-offs between the different language models. As expected, Gemini exhibits the highest performance, reaching near perfect precision and recall on most deceptive patterns. It struggles in two pattern subtypes: hidden-costs and visual interference. In second place is the Qwen model, which struggles in more subtypes. The distilled T5 model exhibits generally acceptable performance, but struggles for pattern categories: interface-interference and obstruction.

6 End-to-End Evaluation

We perform an end-to-end evaluation of *AutoBot* on a real-world dataset, consisting of 1.1K websites, manually annotated by 2 authors (described in Section 6.1). We perform our evaluation with all three LLMs in the *LanguageModule* (see Section 5.1). For this End-to-End Evaluation, our objective is to measure the accuracy of our entire framework (including the vision and language models) in detecting deceptive patterns on websites.

6.1 Dataset

To create the dataset for the end-to-end evaluation, we crawl the websites in the deceptive pattern dataset from Mathur et al. [43]. We use this dataset since it is the latest and most comprehensive dataset of websites with deceptive patterns. We filter out the non-English and offline websites from this dataset, leaving us with 555

在提到的1400个网站中选择网站。接下来，我们抓取了Tranco [51] 列表中排名前1000的网站，使用第5.2.2节中相同的方法过滤掉非英文和NFSW网站，得到597个网站。此外，对于Tranco网站，我们在DuckDuckGo上查询“site:domain”，并以编程方式统计前十个结果页面中每个页面的可交互元素数量。选择元素数量最多的页面进行分析。总的来说，我们的端到端评估数据集中共有1152个网站。

接下来，我们手动标注了这些网站的截图，以识别和定位欺骗性模式，将每个截图与一个元素映射（ElementMap）关联。具体来说，两位作者对随机选取的100个网站进行了标注，并且达到了较高的标注者间一致性（$\kappa = 0.89$） [33]。然后，每位研究人员分别标注了其余的截图。我们将该数据集称为黄金欺骗性设计数据集（D3数据集）。我们在表5中展示了D3数据集的样本分布。

表5：D3 数据集中样本的分布。

类别	亚型	# 样本
不欺骗	不适用	22618
强制行动	强制行动	414
界面干扰	轻推	143
	假稀缺 / 假紧迫	211
	确认羞辱	42
阻塞	视觉干扰	48
	预选	18
偷偷地移动	巧妙措辞	190
	隐性成本	28
	隐藏订阅	379
	伪装广告	306

6.2 研究结果

我们使用 AutoBot 分析 D3 数据集中的网站，并将预测的欺骗模式与真实标注进行比较。

评估结果如表6所示。我们观察到AutoBot框架的性能主要取决于所使用的语言模型类型。这里我们指出，Gemini是我们框架中的教师模型，而Qwen和T5是学生模型，如第5.1节所述。因此，我们看到Gemini的性能最好，其次是Qwen和T5。

我们还评估了Shi等人的 [60] DPGuard框架。为了评估他们的框架，我们首先在他们的分类法和过滤后的分类法之间创建了映射（参见我们的扩展报告 [46]）。在我们的评估中，我们仅考虑可映射到过滤分类法的类别。此外，由于DPGuard框架不提供本地化功能，我们的评估仅侧重于DPGuard识别D3数据集中网页内存在的欺骗性模式的能力。我们的评估显示，DPGuard无法高精度识别欺骗性模式，尤其是在分类‘隐藏订阅’和‘欺骗措辞’时表现困难。这些发现与Shi等人 [60]进行的评估结果一致。

[11]https://tranco-list.eu/list/QGJ74/1000000

websites out of the 1400 sites mentioned. Next, we crawled the top 1000 websites from the Tranco [51] list[11], utilizing the same methodology in Section 5.2.2 to filter out non-English and NFSW websites, resulting in 597 websites. Additionally, for the Tranco websites, we queried “site:domain” on DuckDuckGo and programmatically counted the number of interactable elements for each of the top ten resulting pages. The page with the highest count was selected for analysis. In total, we have 1152 websites in our end-to-end evaluation dataset.

Next, we manually labeled the screenshots of these websites to identify and localize the deceptive patterns, associating each screenshot with an *ElementMap*. Specifically, two authors annotated the same randomly selected 100 websites with high inter-annotator agreement ($\kappa = 0.89$) [33]. Then, each researcher separately labeled the rest of the screenshots. We refer to this dataset as the golden Deceptive Design Dataset (*D3 dataset*). We show the distribution of the samples in *D3 Dataset* in Table 5.

Table 5: Distribution of Samples in *D3 Dataset*.

Category	Subtype	# Samples
Non Deceptive	*Not Applicable*	22618
Forced Action	*Forced Action*	414
Interface Interference	*Nudge*	143
	Fake Scarcity / Fake Urgency	211
	Confirmshaming	42
Obstruction	*Visual Interference*	48
	Pre-Selection	18
Sneaking	*Trick Wording*	190
	Hidden Costs	28
	Hidden Subscription	379
	Disguised Ads	306

6.2 Findings

We analyze the websites in the *D3 Dataset* using *AutoBot* and compare the predicted deceptive patterns to the ground truth annotations.

The results from the evaluation are shown in Table 6. We observe that the performance of the *AutoBot* framework is mainly dependent on the type of language model used. We note here that `Gemini` is the teacher model in our framework, and `Qwen` and `T5` are the student models, as described in Section 5.1. As such, we see that Gemini has the best performance, followed by `Qwen` and `T5`.

We also evaluate Shi et al.'s [60] *DPGuard* framework. To evaluate their framework, we first create a mapping between their taxonomy and the filtered taxonomy (refer to our extended report [46]). For our evaluation, we only consider the categories mappable to the filtered taxonomy. Additionally, as the *DPGuard* framework does not provide localization capabilities, our evaluation is solely focused on *DPGuard*'s ability to identify deceptive patterns present within the webpage in our *D3 Dataset*. Our evaluation shows that *DPGuard* is unable to identify deceptive patterns with high accuracy, especially struggling to classify *'hidden-subscription'* and *'trick-wording'*. These findings are consistent with the evaluation performed by Shi et al. [60].

[11]https://tranco-list.eu/list/QGJ74/1000000

6.2.1 错误分析。在端到端评估中，我们观察到，总体而言，Gemini 在识别欺骗性模式方面表现极佳。然而，对于某些子类型，其表现相对较低。例如，Gemini 在识别“预先选择”（pre-selection）方面的表现有所下降。同样，对于“伪装广告”（disguised-ads），其误报率略高。进一步分析显示，这些误报主要是由于产品植入：通常是类似欺骗性广告的无害内容。例如，一个展示基于模板的购物应用的网站可能会包含用户创建的应用截图，其中一些包含促销文本。虽然这些内容无害，但由于其广告般的外观，常被错误地分类为“伪装广告”此外，通过实证分析，我们观察到 Gemini 在分类 cookie 通知时，常会交替使用“微调”（nudge）和“强制操作”（forced-action）两个类别。

对于较小的模型，如 Qwen 和 T5，我们观察到 AutoBot 有时无法正确识别欺骗模式的类别/子类型。进一步调查错误案例，我们发现存在 AutoBot 错误分类欺骗模式的类别或子类别的情况（尽管正确识别了该模式是欺骗性的）。例如，有些情况下 AutoBot 会错误地将强制行为识别为暗示行为。我们还发现，在 Wikipedia 上，AutoBot 会错误地将非欺骗模式标记为欺骗模式。分类文本中包含与金钱或成本相关的内容，导致模型错误地将文本分类为欺骗措辞。

6.2.2 错误的影响。我们注意到，由于类别或子类别不匹配而导致的误分类对用户的影响是最小的，因为用户仍然会被通知到欺骗性模式。对于误报，用户会收到关于一个实际上不存在的模式的通知。经过进一步检查，用户可以安全地忽略该通知，从而造成最小的干扰。对于漏报——用户的影响可能会很严重。在这种情况下，用户可能会产生虚假的安全感，并因为 AutoBot 的错误而被欺骗性模式所欺骗。我们强调，AutoBot 的高召回率确保这种情况将尽量减少。

7 个应用

我们在三个潜在的下游任务中实例化 AutoBot 框架，每个任务都旨在服务于网络生态系统中的一个利益相关者：用户、开发者，以及监管者和研究人员。

7.1 面向网络用户的浏览器扩展

我们的第一个实例化是一个浏览器扩展，它直接帮助网络用户，这是欺骗性模式的主要目标。该扩展会截取用户活动页面的屏幕截图，并使用 AutoBot 框架进行分析。活动页面可能包含用户的敏感信息。为减少数据隐私问题，该扩展在本地使用 Flan-T5 的精简版（见第 5.2.4 节）执行分析。处理完成后，该扩展会如图 6 所示向用户突出显示欺骗性模式。具体来说，它会在发现欺骗性模式的 UI 元素周围显示边界框，并在用户将鼠标悬停在元素上时通知他们。

6.2.1 Error Analysis. In the end-to-end evaluation, we observe that, overall, `Gemini` performs extremely well in identifying deceptive patterns. However, for certain subtypes, it has comparatively lower performance. For instance, `Gemini` has reduced performance in identifying *'pre-selection'*. Similarly, for *'disguised-ads'*, it has a slightly higher false positive rate. Further analysis shows that these false positives are mainly due to product placement: typically benign content resembling deceptive advertising. For instance, a website showcasing template-based shopping apps may include screenshots of user-created apps, some of which contain promotional text. Although harmless, this text is often misclassified as *'disguised-ads'* due to its advertising-like appearance. Furthermore, through empirical analysis, we have observed `Gemini` interchangeably using *'nudge'* and *'forced-action'*, especially when classifying cookie notices.

For the smaller models, `Qwen` and `T5`, we observe that the *AutoBot* sometimes fails to identify deceptive pattern categories/-subtypes correctly. Investigating the error cases further, we find instances where *AutoBot* misclassified the category or sub-category of the deceptive pattern (while correctly identifying that the pattern is deceptive). For example, there are instances where *AutoBot* incorrectly identifies *forced-action* as *nudge*. We also find that on Wikipedia, *AutoBot* incorrectly tags a non-deceptive pattern as deceptive. The classification text contains money or cost-related text, causing the model to classify the text as *trick-wording* incorrectly.

6.2.2 Impact of Errors. We note that the impact of the misclassifications due to category or sub-category mismatch is minimal on the users as the users will still be notified of a deceptive pattern. For false positives, the user gets notified for a pattern where none exists. Upon further inspection, users can safely ignore the notification, causing minimal distraction. For false negatives - the user impact can be severe. In such cases, users might get into a sense of false security and get deceived by the deceptive pattern because of *AutoBot*'s error. We emphasize that high recall of *AutoBot* ensures that such cases will be minimal.

7 Applications

We instantiate the *AutoBot* framework across three potential downstream tasks, each designed to serve a stakeholder in the web ecosystem: users, developers, and regulators and researchers.

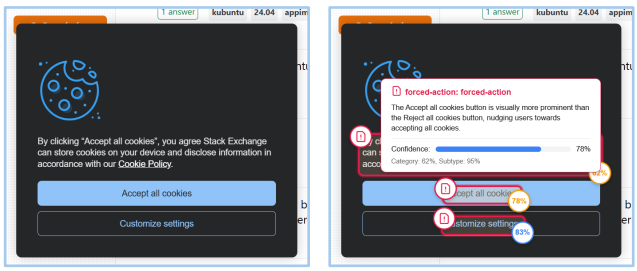
7.1 Browser Extension for Web Users

Our first instantiation is a browser extension that directly helps web users, the primary target of deceptive patterns. The extension takes a screenshot of the user's active page and analyzes it using the *AutoBot* framework. The active page may contain sensitive information of the user. To mitigate data privacy concerns, the extension performs the analysis locally using a distilled version of `Flan-T5` Section 5.2.4. Once processed, the extension highlights deceptive patterns to the users as shown in Figure 6. Specifically, it shows bounding boxes around the UI elements where deceptive patterns are found, and informs the users as they hover over elements.

表6：AutoBot（使用三种底层语言模型：Gemini、Qwen 和 T5）和 DPGu 子类型和二元层面的性能。

	图案类型	精度				
		双子座 Qwen		T5	DPGuard [60]	双子
Category	**不欺骗**	**1.00**	0.98	**1.00**	–	**0.9**
	强制行动	**0.97**	0.89	0.82	–	**0.9**
	界面干扰	**0.89**	0.76	0.55	–	**0.9**
	阻塞	**0.70**	0.57	0.13	–	**0.9**
	偷偷地移动	**0.87**	0.79	0.52	–	**0.9**
Subtype	不适用	**1.00**	0.98	**1.00**	0.78	**0.9**
	确认羞辱	**0.93**	0.75	0.59	0.05	**1.0**
	伪装广告	**0.74**	0.62	0.34	0.49	**0.9**
	假稀缺/假紧迫	**0.94**	0.87	0.68	–	**0.9**
	强制行动	**0.97**	0.89	0.72	0.40	**0.9**
	隐性成本	**0.77**	0.31	0.14	–	**0.9**
	隐藏订阅	**0.93**	0.84	0.51	–	**0.9**
	轻推	**0.79**	0.52	0.10	0.23	**0.8**
	预选	**0.64**	0.64	0.25	0.02	0.9
	巧妙措辞	**0.85**	0.74	0.57	0.00	**0.8**
	视觉干扰	**0.73**	0.54	0.03	0.09	**0.9**
	欺骗性的	**0.90**	0.88	0.72	–	**0.9**
	不欺骗	**1.00**	0.98	0.97	–	**0.9**

–：模型不支持DP分类。



(a) 网站的截图。 (b) AutoBot 运行截图浏览器和检测欺骗性模式燕鸥。

图6：网站截图（左）和 AutoBot 运行截图（右）。

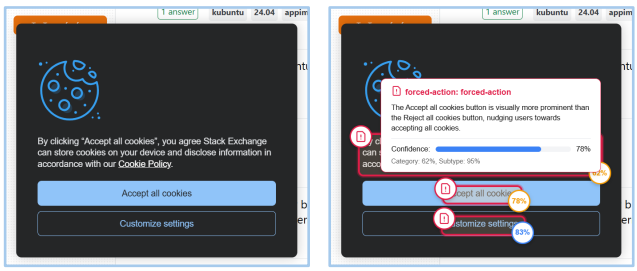
扩展架构。该浏览器扩展采用混合架构，由轻量级浏览器组件和本地运行的 Flask 服务器组成，用于运行 AutoBot 框架，类似于 Zotero 扩展 [20]。在浏览器端，我们使用弹出脚本让用户激活扩展。一旦激活，扩展会截取屏幕并将截图发送到本地 Flask 服务器进行分析。Flask 服务器托管 AutoBot 框架，使用 Flan-T5 作为语言模型。分析完成后，服务器将分类结果返回给扩展，然后由扩展的内容脚本在用户屏幕上呈现。这一呈现如图6所示。

系统级性能。为了使浏览器扩展实用，它必须实时执行分析。因此，我们对其核心的 Flan-T5（语言）和 YOLOv10 Ensemble（视觉）模型进行了延迟测试，使用了三种不同的机器配置，代表不同的硬件能力（现代高端带 GPU、较老的中端、基于 ARM 的），并比较了 CPU 与 GPU 的性能。我们的结果（详细测量请参见我们的扩展报告[46] ）显示，尽管在

Table 6: Performance of *AutoBot* (with three underlying language mode table has three sections, each showing performance at the category, subt

	Pattern Type	Precision				
		Gemini	Qwen	T5	*DPGuard* [60]	Gem
Category	**Non Deceptive**	**1.00**	0.98	**1.00**	–	**0.9**
	Forced Action	**0.97**	0.89	0.82	–	**0.9**
	Interface Interference	**0.89**	0.76	0.55	–	**0.9**
	Obstruction	**0.70**	0.57	0.13	–	**0.9**
	Sneaking	**0.87**	0.79	0.52	–	**0.9**
Subtype	*Not Applicable*	**1.00**	0.98	**1.00**	0.78	**0.9**
	Confirmshaming	**0.93**	0.75	0.59	0.05	**1.0**
	Disguised Ads	**0.74**	0.62	0.34	0.49	**0.9**
	Fake Scarcity / Fake Urgency	**0.94**	0.87	0.68	–	**0.9**
	Forced Action	**0.97**	0.89	0.72	0.40	**0.9**
	Hidden Costs	**0.77**	0.31	0.14	–	**0.9**
	Hidden Subscription	**0.93**	0.84	0.51	–	**0.9**
	Nudge	**0.79**	0.52	0.10	0.23	**0.8**
	Pre-Selection	**0.64**	0.64	0.25	0.02	0.9
	Trick Wording	**0.85**	0.74	0.57	0.00	**0.8**
	Visual Interference	**0.73**	0.54	0.03	0.09	**0.9**
	Deceptive	**0.90**	0.88	0.72	–	**0.9**
	Not Deceptive	**1.00**	0.98	0.97	–	**0.9**

–: the DP classification is not supported by the model.



(a) Screenshot of a website. (b) Screenshot of *AutoBot* running on browser and detecting deceptive patterns.

Figure 6: Screenshot of website (left) and *AutoBot* running (right).

Extension Architecture. The browser extension utilizes a hybrid architecture consisting of lightweight browser components with a locally running Flask server to run the *AutoBot* framework, similar to the *Zotero* extension [20]. On the browser side, we use a popup script to allow users to activate the extension. Once active, the extension takes a screenshot and sends it to the local Flask server for analysis. The Flask server hosts the *AutoBot* framework, using `Flan-T5` as the language model. After analysis, the server returns the classifications to the extension, which are then rendered by the content script of the extension on the user's screen. This rendering is shown in Figure 6.

System Level Performance. For the browser extension to be practical, it must perform its analysis in real time. We, therefore, conducted latency tests on its core `Flan-T5` (Language) and `YOLOv10 Ensemble` (Vision) models using three different machine configurations representing various hardware capabilities (modern high-end with GPU, older mid-range, ARM-based) and comparing CPU versus GPU performance. Our results (refer to our extended report [46] for detailed measurements) show that while performance on

PGuard [60] 在 D3 数据集上的性能。该表分为三个部分，分别显示类别、

回忆				F1 分数			
双子座 Qwen		T5	DPGuard [60]	双子座 Qwen		T5	*DPGuard* [60]
0.99	**0.99**	0.96	–	**0.99**	**0.99**	0.98	–
0.94	0.85	0.83	–	**0.95**	0.87	0.82	–
0.95	0.64	0.78	–	**0.92**	0.69	0.64	–
0.97	0.71	0.93	–	**0.81**	0.63	0.23	–
0.94	0.73	0.85	–	**0.90**	0.76	0.64	–
0.99	**0.99**	0.91	0.74	**0.99**	**0.99**	0.95	0.76
1.00	0.69	0.85	0.32	**0.97**	0.72	0.70	0.09
0.95	0.61	0.81	0.62	**0.83**	0.61	0.48	0.55
0.96	0.71	0.89	–	**0.95**	0.78	0.77	–
0.94	0.85	0.84	0.51	**0.95**	0.87	0.77	0.45
0.91	0.28	0.38	–	**0.83**	0.29	0.21	–
0.96	0.72	0.83	–	**0.95**	0.78	0.63	–
0.89	0.43	0.48	0.58	**0.83**	0.47	0.17	0.33
0.94	0.94	**1.00**	0.14	**0.76**	**0.76**	0.40	0.03
0.82	0.69	0.67	0.00	**0.83**	0.71	0.62	0.00
0.98	0.62	0.93	0.22	**0.84**	0.58	0.05	0.13
0.97	0.81	0.95	–	**0.93**	0.84	0.82	–
0.99	**0.99**	0.97	–	**0.99**	**0.99**	0.98	–

使用仅 CPU 的较老硬件每个模块大约需要 1.5 到 3 秒（例如，在仅 CPU 的 i5 笔记本上，YOLOv10 集成模型需要 1.95 秒），而配备 GPU 的新设备几乎可以实现实时性能。具体来说，在 2023 年的 GPU 笔记本上，T5 推理（输入 800 个 token）耗时不到 0.5 秒，而 YOLOv10 集成模型的分类时间不到 0.3 秒。我们的实验表明，将 AutoBot 实现为浏览器扩展是可行的，因为支持 GPU 的现代硬件能够提供响应迅速的体验而不会有显著的性能延迟。

7.2 面向 Web 开发者的 Lighthouse 报告

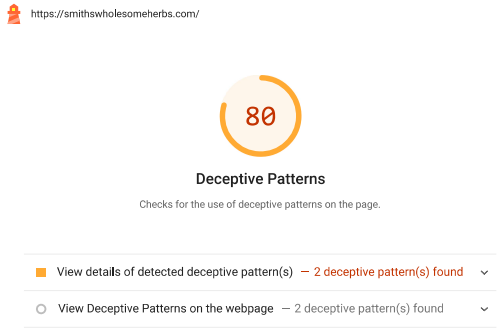


图 7：Lighthouse 报告上的自定义审核。

研究表明，尽管开发者通常没有意识到自己网站上的欺骗性模式及其对用户的影响，但如果得到适当的告知，他们愿意解决这些模式 [66]。为此，我们将 AutoBot 集成到 Lighthouse [12] [25]，——这是开发者用来获取网站自动审计的流行工具

[12]https://chromewebstore.google.com/detail/lighthouse/blipmdconlkpinefehnmjammfjpmpbjk

odels: Gemini, Qwen, and T5) and *DPGuard* [60] on the *D3 Dataset*. The subtype, and binary levels.

Recall				F1-Score			
emini	Qwen	T5	*DPGuard* [60]	Gemini	Qwen	T5	*DPGuard* [60]
0.99	**0.99**	0.96	–	**0.99**	**0.99**	0.98	–
0.94	0.85	0.83	–	**0.95**	0.87	0.82	–
0.95	0.64	0.78	–	**0.92**	0.69	0.64	–
0.97	0.71	0.93	–	**0.81**	0.63	0.23	–
0.94	0.73	0.85	–	**0.90**	0.76	0.64	–
0.99	**0.99**	0.91	0.74	**0.99**	**0.99**	0.95	0.76
1.00	0.69	0.85	0.32	**0.97**	0.72	0.70	0.09
0.95	0.61	0.81	0.62	**0.83**	0.61	0.48	0.55
0.96	0.71	0.89	–	**0.95**	0.78	0.77	–
0.94	0.85	0.84	0.51	**0.95**	0.87	0.77	0.45
0.91	0.28	0.38	–	**0.83**	0.29	0.21	–
0.96	0.72	0.83	–	**0.95**	0.78	0.63	–
0.89	0.43	0.48	0.58	**0.83**	0.47	0.17	0.33
0.94	0.94	**1.00**	0.14	**0.76**	**0.76**	0.40	0.03
0.82	0.69	0.67	0.00	**0.83**	0.71	0.62	0.00
0.98	0.62	0.93	0.22	**0.84**	0.58	0.05	0.13
0.97	0.81	0.95	–	**0.93**	0.84	0.82	–
0.99	**0.99**	0.97	–	**0.99**	**0.99**	0.98	–

older hardware using only the CPU takes roughly **1.5 to 3 seconds** per module (e.g., **1.95 seconds** for `YOLOv10 Ensemble` on a CPU-only i5 laptop), newer devices with GPUs achieve near real-time performance. Specifically, on the 2023 laptop with a GPU, `T5` inference (with 800 tokens as input) took less than **0.5 seconds**, and `YOLOv10 Ensemble` performed its classifications in less than **0.3 seconds**. Our experiments show the feasibility of implementing *AutoBot* as a browser extension, as modern hardware with GPU support enables a responsive experience without major performance delays [2, 30].

7.2 Lighthouse Reports for Web Developers

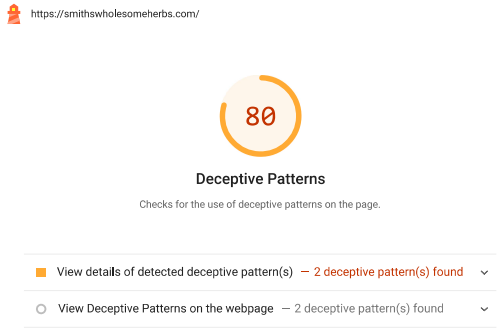


Figure 7: Custom Audit on a Lighthouse Report.

Research shows that while developers are often unaware of deceptive patterns on their websites and their impacts on users, they are open to addressing these patterns if properly informed [66]. To that end, we integrate *AutoBot* into Lighthouse [12] [25], a popular tool developers use to receive automated audits on website

[12]https://chromewebstore.google.com/detail/lighthouse/blipmdconlkpinefehnmjammfjpmpbjk

质量。谷歌将此工具与 Chrome 开发者工具捆绑在一起，主要平台如 Shopify、Wix 和 Squarespace 将其集成到他们的工作流程中。我们使用 AutoBot 创建了一个自定义的 Lighthouse 审核，以直接适应网页开发者的工作流程。此审核使用现有的 Lighthouse 收集器捕获网站截图，通过 AutoBot 处理，并最终将结果整合到 Lighthouse 报告中，如图 7 所示。

我们将 Lighthouse 与我们的自定义审计打包在一个 Docker 容器中，以提高可用性。该 Docker 容器保持了 Lighthouse 的所有功能，同时添加了我们的自定义审计。该容器只需将 URL 作为输入即可运行完整的审计，并向开发者提供报告。报告根据页面上发现的欺骗性模式数量（n）为开发者提供欺骗性模式得分，评分方案如下：

$$S(n) = \begin{cases} 100, & \text{if } n = 0 \\ 89, & \text{if } n = 1 \\ \max(100 - 10n, 0), & \text{if } n > 1 \end{cases}$$

评分方案与欺骗模式的数量成反比。它为单一的欺骗模式分配0.89分，以确保审计显示为失败状态。开发者看到的 Lighthouse 报告示例如图 7 所示。

7.3 启用网络规模分析

我们最后一次实现的 AutoBot 框架是一个用于可扩展网站分析的工具。该工具为研究人员和监管者提供服务，为他们提供必要的自动化功能，以调查更广泛的网络欺骗行为。它从一个 URL 列表中获取输入，然后爬取每个 URL，截取截图，从每个截图中提取元素映射（ElementMap），并将元素映射传递给语言模块。我们在该工具中使用带批处理 API 的 Gemini 模型，以低成本生成准确的分析。

在 Shopify 和 Tranco 网站上的测量。我们使用该工具分析了 11,118 个网站，其中包括从 Tranco 列表 [51]，中选择的 6,626 个多样化的热门域名，以及来自 Shopify 合作伙伴目录 [15] 的 4,492 个电子商务网站。我们在第 5.2.2 节中更详细地说明了网站选择过程。

在图8中展示了已识别的欺骗模式在两个领域的完整分布。我们观察到，偷偷行动（sneaking）是两个领域中最普遍的欺骗模式。在 Shopify 网站上，第二常见的是界面干扰（interface-interference）。例如，电子商务网站倾向于使用虚假的稀缺性或紧迫感（例如，“限时优惠：接下来的 5 分钟内享受 20% 折扣”），这被归类为虚假稀缺-虚假紧迫（fake-scarcity-fake-urgency）。在 Tranco 网站上，强制操作（forced-action）是继偷偷行动之后的下一个最常见的模式。我们的分析还显示，许多网站同时采用多种不同类别的欺骗模式。例如，bedbathandbeyond.com（参见我们的扩展报告 [46]）包含“强制操作”（通过没有提供明确的拒绝 Cookie 选项）。

[13]https://www.shopify.com/, https://www.wix.com/, https://www.squarespace.com/[14] https://github.com/GoogleChrome/lighthouse/blob/main/core/gather/gatherers/full-page-screenshot.js [15] https://www.shopify.com/partners/directory

quality. Google bundles this tool with ChromeDev Tools, and major platforms such as Shopify, Wix, and Squarespace[13] integrate it into their workflows. We created a custom Lighthouse audit using *AutoBot* to fit directly into a web developer's workflow. This audit uses an existing `Lighthouse Gatherer` [14] to capture screenshots of the website, processes it using *AutoBot*, and finally incorporates the findings into a Lighthouse report, as shown in Figure 7.

We package Lighthouse with our custom audit in a docker container for better usability. The docker container maintains all functionality of Lighthouse while adding our custom audit. The container simply takes an URL as input to run the entire audit and give the developer a report. The report provides the developer with *DeceptivePattern Score* based on the number of deceptive patterns (n) found on the page using the following scoring scheme:

$$S(n) = \begin{cases} 100, & \text{if } n = 0 \\ 89, & \text{if } n = 1 \\ \max(100 - 10n, 0), & \text{if } n > 1 \end{cases}$$

The scoring scheme scales inversely with the number of deceptive patterns. It assigns a score of 0.89 for a single deceptive pattern to ensure that the audit shows a failure condition. An example of the Lighthouse Report as a developer would see is shown in Figure 7.

7.3 Enabling Web-Scale Analysis

Our last instantiation of the *AutoBot* framework is a tool designed for scalable website analysis. This tool serves researchers and regulators, providing them with the necessary automated capabilities to investigate the broader landscape of online deceptive practices. It takes input from a list of URLs. Then, it crawls each URL, takes a screenshot, extracts the *ElementMap* from each screenshot, and passes the *ElementMap* to the Language Module. We use the Gemini model with batch API in this tool to generate accurate analysis at a low cost.

Measurement on Shopify and Tranco Websites. We use this tool to analyze 11,118 websites, consisting of 6,626 diverse, popular domains selected from the Tranco list [51], and 4,492 e-commerce sites from the *Shopify Partners Directory*[15]. We detail our website selection process earlier in Section 5.2.2.

The complete distribution of the identified deceptive patterns across the two domains is in Figure 8. We observe that *sneaking* is the most prevalent deceptive pattern across both domains. On Shopify websites, the second prevalent is *interface-interference*. For example, e-commerce websites tend to use fake scarcity or urgency (e.g., "Limited time offer: get 20% off for next 5 min"), which is classified as *fake-scarcity-fake-urgency*. On Tranco websites, *forced-action* is the next most prevalent pattern after *sneaking*. Our analysis also reveals that many websites employ multiple distinct categories of deceptive patterns simultaneously. For example, bedbathandbeyond.com (refer to our extended report [46]) consists of *'forced-action'* (by providing no clear option to reject cookies)

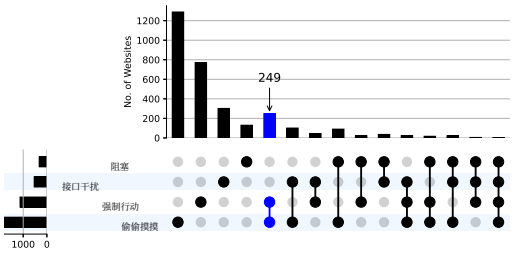
[13]https://www.shopify.com/, https://www.wix.com/, https://www.squarespace.com/

[14]https://github.com/GoogleChrome/lighthouse/blob/main/core/gather/gatherers/full-page-screenshot.js

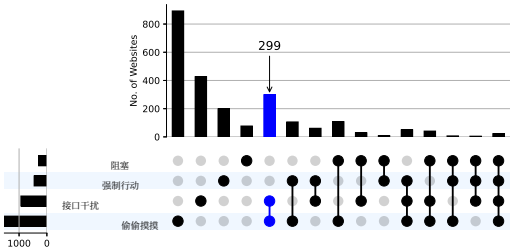
[15]https://www.shopify.com/partners/directory

107

同时伴有“阻碍”（通过以过小字体展示邮件列表条款和条件）



(a) AutoBot 在 Tranco 网站上识别的欺骗模式分布



(b) AutoBot在Shopify网站上识别的欺骗性模式分布

图8：各种领域中欺骗性模式的分布，例如 (a) 访问量最高的 Tranco 网站 和 (b) 基于 Shopify 的电子商务网站

8 用户研究

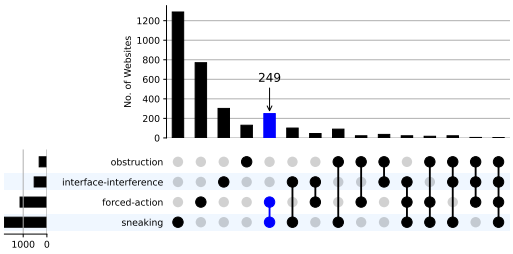
我们进行了两项评估：一项网站可用性评估，以评估高亮显示对网站的影响；另一项是开发者外展计划，我们联系了 Shopify 开发者，讨论他们网站的 Lighthouse 报告（见第7.3节）。

8.1 网站可用性评估

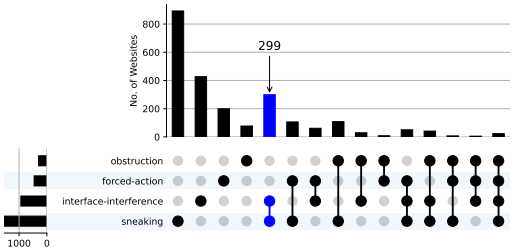
我们对原型网站进行了网站可用性测试，以评估突出显示欺骗性模式如何影响网站的可用性。我们从Prolific招募了151名美国参与者，他们完成任务的中位时间为7分钟，并获得了2美元的报酬。我们没有收集人口统计数据，但指示Prolific确保在其人口统计类别中均匀分布。我们机构的伦理审查委员会（IRB）确定，根据DHHS和FDA的规定，本研究不构成人类受试者研究。

8.1.1 研究设计。我们进行了一个被试内研究，参与者访问了两个定制网站。一个版本突出显示了欺骗性模式以模拟我们的浏览器扩展程序，而另一个版本则没有。在每次交互后，参与者完成了系统可用性量表（SUS）问卷 [12]。在每个网站上，参与者执行四项任务中的一项：注册、下载文件、购买商品或阅读新闻文章。当参与者将鼠标悬停在高亮文本上时，会出现一个横幅，提醒他们注意欺骗性模式。这些定制网站基于 UXP 的真实示例[2]。

alongside ‘*obstruction*’ (by presenting mailing list terms and conditions in an excessively small font).



(a) Distribution of Deceptive Patterns identified by *AutoBot* on Tranco websites



(b) Distribution of Deceptive Patterns identified by *AutoBot* on Shopify Websites

Figure 8: Distribution of deceptive patterns across various domains such as (a) Most visited Tranco websites and (b) Shopify based e-commerce websites

8 User Studies

We conducted two evaluations: a website usability evaluation to assess the effects of highlights on a website, and a developer outreach program where we contacted Shopify developers to discuss their site's Lighthouse report (see Section 7.3).

8.1 Website Usability Evaluation

We conducted a website usability test with prototyped websites to evaluate how highlighting deceptive patterns affects website usability. We recruited 151 U.S.-based participants from Prolific, who were compensated $2 for a task that took a median of 7 minutes to complete. We did not collect demographic data but instructed Prolific to ensure an even distribution across its demographic categories. Our institution's IRB determined this study does not constitute research involving human subjects under DHHS and FDA regulations.

8.1.1 Study Design. We conducted a within-subjects study in which participants visited two custom-made websites. One version featured highlighted deceptive patterns to simulate our browser extension, while the other did not. After each interaction, participants completed a System Usability Scale (SUS) questionnaire [12]. On each site, participants performed one of four tasks: signing up, downloading a file, shopping for an item, or reading a news article. When a participant hovered over highlighted text, a banner appeared, cautioning them about the deceptive pattern. These custom websites were based on real-world examples from *UXP*[2]

107

暗黑模式[16]（有关示例，请参阅我们扩展报告 [46]）。参与者直接与它们互动，无需安装浏览器扩展。在完成任务后，参与者回答了一份后续研究问卷，询问：（1）关于欺骗性模式的提示是否有用，（2）突出显示是否帮助他们注意到这些模式，以及（3）突出显示是否影响了他们的选择。

8.1.2 发现。对系统可用性量表（SUS）得分进行的Wilcoxon符号秩检验显示，有高亮与无高亮的网站在可用性方面没有统计学显著差异（$p = 0.106$），支持我们的零假设。然而，我们观察到高亮网站的平均SUS得分高出2分。参与者反馈显示，大多数人认为高亮（65.5%）和提示（56.3%）对于识别欺骗性模式有帮助，38%的人报告说高亮会影响他们的行为。

8.1.3 未来研究。我们的研究结果表明，突出显示欺骗性模式可以帮助用户识别这些模式，同时不影响网站的可用性。然而，我们的评估并未衡量高亮显示对用户行为的影响。未来的工作应在两个关键方面解决这一限制。首先，研究应探讨高亮显示如何影响用户的选择和偏好。其次，需要使用不同版本的 AutoBot 浏览器扩展进行真实环境评估，以评估其性能、可用性和实用性之间的实际权衡。这些研究可以让参与者安装扩展，并根据他们在真实网站上的体验提供反馈。

8.2 开发者推广

我们分析了4,492个Shopify网站（见第7.3节）并为每个网站生成了Lighthouse报告。接下来，我们将各自网站的Lighthouse报告发送给开发者，并告知他们网站上发现的欺骗性模式。

8.2.1 通知流程。为了选择要分析的网站和要联系的开发者，我们首先爬取 Shopify 合作伙伴目录，以识别开发者及其最受欢迎的网站。接下来，我们分析了这些网站的欺骗性模式（见第 7.3 节），为每个网站生成了定制的 Lighthouse 报告，并通过电子邮件通知开发者其网站上可能存在的欺骗性模式，并征求他们对 Lighthouse 报告的反馈。具体来说，我们问了他们四个问题：（1）报告是否有用？（2）他们希望报告包含更多还是更少的信息？（3）根据报告，他们是否愿意进行任何更改？（4）他们是否对一个可以在整个网站上运行此分析的工具感兴趣？

8.2.2 伦理考虑。在我们的电子邮件中，我们自我介绍为正在开发一个自动检测欺骗模式系统的研究人员。我们解释说，我们的目标是向他们通报其网站上潜在的欺骗模式，并了解他们对生成的Lighthouse报告的看法，同时澄清参与是自愿的，且不会收集任何可识别个人身份的信息（PII）。由于没有收集PII，我们机构的伦理审查委员会（IRB）将此次外展认定为“非人体受试者研究”，因此免于事先获得同意的要求。

[16]https://darkpatterns.uxp2.com/

Dark Patterns[16](refer to our extended report [46] for examples). Participants interacted with them directly, without installing a browser extension. After the tasks, participants answered a post-study questionnaire asking if: (1) the hints about deceptive patterns were useful, (2) the highlights helped them notice the patterns, and (3) the highlights influenced their choice.

8.1.2 Findings. A Wilcoxon signed-rank test of the System Usability Scale (SUS) scores revealed no statistically significant difference in usability between websites with and without highlights ($p = 0.106$), supporting our null hypothesis. However, we observe a 2-point higher mean SUS score for the highlighted website. Participant feedback showed that most found the highlights (65.5%) and hints (56.3%) helpful for recognizing deceptive patterns, and 38% reported that the highlights would influence their behavior.

8.1.3 Future Studies. Our findings suggest that highlighting deceptive patterns can help users recognize them without compromising the website's usability. However, our evaluation did not measure the impact of highlights on user behavior. Future work should address this limitation in two key areas. First, studies should explore how the highlights influence user choice and preferences. Second, a real-world evaluation using different versions of the *AutoBot* browser extension is needed to assess the practical trade-offs between its performance, usability, and utility. These studies could involve participants installing the extension and provide feedback based on their experience on real websites.

8.2 Developer Outreach

We analyzed 4,492 Shopify websites (see Section 7.3) and generated Lighthouse reports for each. Next, we contacted the developers with the Lighthouse reports of their website and informed them about the deceptive patterns found on their site.

8.2.1 Notification Process. To choose the websites to analyze and the developers to contact, we start by crawling the *Shopify Partners Directory* to identify developers and their most popular websites. Next, we analyzed these sites for deceptive patterns (see Section 7.3), generated a custom Lighthouse report for each one, and emailed the developers to inform them of potential deceptive patterns on their sites and get their feedback on the Lighthouse reports. Specifically, we asked them four questions: (1) Was the report useful? (2) Would they like more or less information included? (3) In light of the report, would they be willing to make any changes? and (4) Would they be interested in a tool to run this analysis on their entire website?

8.2.2 Ethical Consideration. In our emails, we identified ourselves as researchers developing a system to automatically detect deceptive patterns. We explained that our goal was to inform them of potential deceptive patterns on their sites and understand their perspective on the generated Lighthouse reports, and clarified that participation was optional and no personally identifiable information (PII) was being collected. Because no PII was collected, our institution's IRB certified this outreach as "not human subject research," exempting it from requiring prior consent.

[16]https://darkpatterns.uxp2.com/

8.2.3 结果。3 位开发者回复了我们的电子邮件。两位开发者表示，移除 DP 是超出其权限的所有者级别决策。第三位受访者以客户保留为理由没有实施更改。

9 限制

可检测模式的范围。AutoBot 的一个限制来自静态用户界面分析。我们的方法本质上将可检测的欺骗模式范围限制在特定时间单页上可见的模式。诸如烦扰（nagging）等欺骗模式无法通过这种方法检测，因此，我们筛选了 Gary 等人的分类法 [27]，以专注于静态欺骗模式。

硬件限制。另一项实际限制是由于使用 T5 部署 AutoBot 扩展时的硬件限制。该模型大约需要 1GB 的内存，而旧设备可能无法轻松提供。此外，我们的实验表明，虽然在现代硬件上可以实现接近实时的延迟，但缺乏 GPU 加速可能会显著影响延迟。

多语言支持。AutoBot目前仅限于英文网站，因为蒸馏管道中存在特定语言的数据集。然而，我们注意到，通过使用多语言模型并策划特定语言的蒸馏数据集，可以将AutoBot扩展到其他语言。

YOLOv10 集成标签集。最终，YOLOv10 集成仅限于识别一组 7 个常见的 UI 元素。其他在现代网站上常见的交互元素，例如滑块和日期选择器，模型并未明确识别。虽然这可能会减少语言模块可用的上下文信息，但我们通过实验证实，欺骗性模式很少使用其他交互式 UI 元素。

10 未来工作

AutoBot 是一个框架，可以截取屏幕并返回本地化的欺骗模式。我们提供了三个基于它构建的示例应用。我们设想像 lighthouse-ci[17] 这样的应用可以轻松扩展我们的工作，将 AutoBot 集成到开发者的 CI/CD 工作流程中。监管机构也可以使用与其法规对齐的模型来自动执行政策。

未来工作的另一个方向是增强 AutoBot 的多语言能力。虽然欺骗性模式与语言无关，但当前框架的训练和评估数据集主要集中在英语网站上。未来的工作可以探索针对特定语言的 LanguageModule 版本，或者整合能够跨多种语言进行推理的高级大型语言模型（LLM）。

最后，我们还设想将 AutoBot 用作生成大规模网站数据集的工具，其中的元素会自动标注为欺骗模式。对这个任务进行 VLLM 微调/再训练的一个重大瓶颈是缺乏大规模标注的训练数据。使用 AutoBot 创建的数据集可以用来克服这一挑战，并有可能在未来提升 VLLM 在检测欺骗模式方面的性能。

[17]https://github.com/GoogleChrome/lighthouse-ci

8.2.3 Outcome. 3 developers replied to our emails. Two developers indicated that removing DPs was an owner-level decision beyond their authority. The third respondent cited customer retention as the reason for not implementing changes.

9 Limitations

Scope of Detectable Patterns. A limitation for *AutoBot* comes from the static UI analysis. Our approach inherently restricts the scope of detectable deceptive patterns to those visually present on a single page at a given time. Deceptive patterns such as *nagging* cannot be detected through this approach, and thus, we filter Gary et. al.’s taxonomy [27] to focus on static deceptive patterns.

Hardware Constraints. Another practical limitation arises due to hardware limitations associated with deploying *AutoBot*’s extension using `T5`. The model requires approximately 1GB of memory, which might not be easily available on older devices. Furthermore, our experiments show that while near-real-time latency is achievable on modern hardware, the lack of GPU acceleration can significantly affect latency.

Multilingual Support. AutoBot’s is currently limited to English-only websites due to the presence of language-specific datasets in the distillation pipeline. However, we note that it is possible to extend *AutoBot* to other languages by using a multi-lingual language models, and curating a language specific distillation dataset.

YOLOv10 Ensemble Label Set. Finally, the *YOLOv10 Ensemble* is limited to identifying only a set of 7 common UI elements. Other interactive elements prevalent on modern websites, such as sliders and date pickers, are not explicitly recognized by the model. While this could potentially reduce the contextual information available to the *Language Module*, we have empirically observed that deceptive patterns rarely use other interactive UI elements.

10 Future Work

AutoBot is a framework that takes a screenshot and returns localized deceptive patterns. We provide three sample applications that build on top of it. We envision that applications like `lighthouse-ci`[17] can easily extend our work to integrate *AutoBot* into developer CI/CD workflows. Regulators can also use *AutoBot* with models aligned to their regulations to enforce policies automatically.

Another direction for future work is to enhance *AutoBot*’s multilingual capabilities. While deceptive patterns are language-agnostic, the current framework’s training and evaluation datasets were focused on English-language websites. Future efforts could explore language-specific versions of the *LanguageModule* or integrate advanced LLMs that can reason across multiple languages.

Lastly, we also envision using *AutoBot* as a tool to generate large-scale datasets of websites with elements automatically labeled for deceptive patterns. A significant bottleneck for finetuning/retraining a VLLM for this task is the lack of large-scale annotated training data. Datasets created using *AutoBot* can be utilized to overcome this challenge and potentially improve the performance of VLLMs in detecting deceptive patterns in the future.

[17]https://github.com/GoogleChrome/lighthouse-ci

11 结论

在本文中，我们介绍了 AutoBot，这是一种用于自动检测网站上欺骗性模式的框架。AutoBot 采用模块化的方法：首先，它捕获网站的截图，并使用视觉模块对其进行处理，以提供网站的文本表示（ElementMap）。然后，它使用语言模块分析 ElementMap，以识别欺骗性模式及其类型。我们在一个真实世界网站的数据集上评估 AutoBot，以展示其在识别和定位欺骗性模式方面的准确性。随后，我们在三种设置中实例化了 AutoBot：面向用户的浏览器扩展、面向开发者的 Lighthouse 报告以及面向研究人员/监管人员的网站分析工具。

致谢

这项工作得到了国家科学基金会（NSF）通过奖项CNS-1942014和CNS-2247381的支持，以及来自谷歌PSS隐私教师奖计划的研究资助。最后，我们感谢评审专家的深思熟虑的建议。

参考文献

[1] 欧洲议会和理事会于2016年4月27日通过的关于保护自然人个人数据处理及其自由流通的第(EU) 2016/679号条例，并废止指令95/46/EC（通用数据保护条例）。https://eur-lex.europa.eu/eli/reg/2016/679/oj。访问日期: 2024-09-02。[2] 2024 WebGPU. https://www.w3.org/TR/webgpu/ 定义了用于网络的现代图形和计算API。[3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat 等. 2023. GPT-4 技术报告. arXiv 预印本 arXiv:2303.08774 (2023). [4] Juris Hannah Adorna, Aurel Jared Dantis, Rommel Feria, Ligaya Leah Figueroa, 和 Rowena Solamo. 2024. 开发用于自动检测 Cookie 横幅中欺骗模式的浏览器扩展. 收录于计算：理论与实践研讨会 (WCTP 2023) 论文集, 第20卷. Springer

11 Conclusion

In this paper, we introduce *AutoBot*, a framework to automatically detect deceptive patterns on websites. *AutoBot* employs a modular approach: first, it captures a screenshot of the website and processes it using the *Vision Module* to provide a textual representation of the website (*ElementMap*). It then analyzes the *ElementMap* using a *Language Module* to identify deceptive patterns and their type. We evaluate *AutoBot* on a dataset of real-world websites to demonstrate its accuracy in identifying and localizing deceptive patterns. We then instantiate *AutoBot* in three settings: a user-facing browser extension, a developer-facing Lighthouse report, and a researcher/regulator-facing website analysis tool.

Acknowledgments

This work was supported by the NSF through awards CNS-1942014 and CNS-2247381, and by a research grant from the Google PSS Privacy Faculty Award program. Finally, we thank the reviewers for their thoughtful recommendations.

References

[1] 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). https://eur-lex.europa.eu/eli/reg/2016/679/oj. Accessed: 2024-09-02.

[2] 2024. WebGPU. https://www.w3.org/TR/webgpu/ Defines a modern graphics and compute API for the Web..

[3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[4] Juris Hannah Adorna, Aurel Jared Dantis, Rommel Feria, Ligaya Leah Figueroa, and Rowena Solamo. 2024. Developing a Browser Extension for the Automated Detection of Deceptive Patterns in Cookie Banners. In *Proceedings of the Workshop on Computation: Theory and Practice (WCTP 2023)*, Vol. 20. Springer Nature, 101.

[5] Vibhor Agarwal, Nakul Thureja, Madhav Krishan Garg, Sahiti Dharmavaram, Dhruv Kumar, et al. 2024. " Which LLM should I use?": Evaluating LLMs for tasks performed by Undergraduate Computer Science Students in India. *arXiv e-prints* (2024), arXiv–2402.

[6] Spike aka Steve Spiker. 2023. Tweet by Spike aka Steve Spiker on X. https://x.com/spjika/status/1686492710910427137. Accessed: 2024-09-02.

[7] Anthropic. 2024. Fine-tune Claude 3 Haiku in Amazon Bedrock. https://www.anthropic.com/news/fine-tune-claude-3-haiku. https://www.anthropic.com/news/fine-tune-claude-3-haiku Accessed: April 15, 2025.

[8] Vinayshekhar Bannihatti Kumar, Roger Iyengar, Namita Nisal, Yuanyuan Feng, Hana Habib, Peter Story, Sushain Cherivirala, Margaret Hagan, Lorrie Cranor, Shomir Wilson, Florian Schaub, and Norman Sadeh. 2020. Finding a Choice in a Haystack: Automatic Extraction of Opt-Out Statements from Privacy Policy Text. In *Proceedings of The Web Conference 2020*. ACM, Taipei Taiwan, 1943–1954. doi:10.1145/3366423.3380262

[9] Christoph Bösch, Benjamin Erb, Frank Kargl, Henning Kopp, and Stefan Pfattheicher. 2016. Tales from the dark side: Privacy dark strategies and privacy dark patterns. *Proceedings on Privacy Enhancing Technologies* (2016).

[10] Harry Brignull. [n. d.]. Deceptive Patterns. https://www.deceptive.design/.

[11] H Brignull, M Leiser, C Santos, and K Doshi. 2023. Deceptive patterns – user interfaces designed to trick you. https://www.deceptive.design/

[12] J Brooke. 1996. SUS: A quick and dirty usability scale. *Usability Evaluation in INdustry/Taylor and Francis* (1996).

[13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[14] Thomas Cairns. 2021. extcolors: Extract colors from an image using k-means clustering. https://pypi.org/project/extcolors/. Accessed: 2024-09-02.

[15] California Privacy Protection Agency. 2024. California Privacy Rights Act (CPRA). https://thecpra.org/. Accessed: 2024-09-04.

[16] Jieshan Chen, Jiamou Sun, Sidong Feng, Zhenchang Xing, Qinghua Lu, Xiwei Xu, and Chunyang Chen. 2023. Unveiling the tricks: Automated detection of dark patterns in mobile applications. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–20.

109

[17] 陈克勤, 张朝, 曾威利, 张日聪, 朱锋, 赵瑞. 2023. Shikra: 释放多模态大语言模型的指称对话魔力. arXiv预印本 arXiv:2306.15195 (2023). [18] 郑亨远, 侯乐, Shayne Longpre, Barret Zoph, 易泰, William Fédus, 李云轩, 王雪志, Mostafa Dehghani, Siddhartha Brahma, 等. 2024. 扩展指令微调语言模型. 《机器学习研究杂志》 25, 70 (2024), 1–53. [19] Gregory Conti 和 Edward Sobiesk. 2010. 恶意接口设计：利用用户. 在第19届国际万维网会议论文集中. 271–280. [20] 数字学术公司. 2024. Zotero. 弗吉尼亚州维也纳. https://www.zotero.org/ [21] 崔浩, Zahra Shamsi, Gowoon Cheon, 马学间, 李书彤, Maria Tikhanovskaya, Peter Norgaard, Nayantara Mudur, Martyna Plomecka, Paul Raccuglia, 等. 2025. CURIE: 在多任务科学长文本理解与推理中评估大语言模型. arXiv预印本 arXiv:2503.13

[17] Keqin Chen, Zhao Zhang, Weili Zeng, Richong Zhang, Feng Zhu, and Rui Zhao. 2023. Shikra: Unleashing multimodal llm's referential dialogue magic. *arXiv preprint arXiv:2306.15195* (2023).

[18] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.

[19] Gregory Conti and Edward Sobiesk. 2010. Malicious interface design: exploiting the user. In *Proceedings of the 19th international conference on World wide web*. 271–280.

[20] Corporation for Digital Scholarship. 2024. *Zotero*. Vienna, VA. https://www.zotero.org/

[21] Hao Cui, Zahra Shamsi, Gowoon Cheon, Xuejian Ma, Shutong Li, Maria Tikhanovskaya, Peter Norgaard, Nayantara Mudur, Martyna Plomecka, Paul Raccuglia, et al. 2025. CURIE: Evaluating LLMs On Multitask Scientific Long Context Understanding and Reasoning. *arXiv preprint arXiv:2503.13517* (2025).

[22] Andrea Curley, Dympna O'Sullivan, Damian Gordon, Brendan Tierney, and Ioannis Stavrakakis. 2021. The Design of a framework for the detection of web-based dark patterns. (2021).

[23] Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. 2024. Molmo and pixmo: Open weights and open data for state-of-the-art multimodal models. *arXiv preprint arXiv:2409.17146* (2024).

[24] Esraa Abu Elsoud, Mohamad Hassan, Omar Alidmat, Esraa Al Henawi, Nawaf Alshdaifat, Mosab Igtait, Ayman Ghaben, Anwar Katrawi, and Mohmmad Dmour. 2024. Under Sampling Techniques for Handling Unbalanced Data with Various Imbalance Rates: A Comparative Study. *International Journal of Advanced Computer Science & Applications* 15, 8 (2024).

[25] Google Chrome Developers. [n. d.]. *Lighthouse Overview*. https://developer.chrome.com/docs/lighthouse/overview

[26] Colin M Gray, Yubo Kou, Bryan Battles, Joseph Hoggatt, and Austin L Toombs. 2018. The dark (patterns) side of UX design. In *Proceedings of the 2018 CHI conference on human factors in computing systems*. 1–14.

[27] Colin M Gray, Cristiana Santos, and Nataliia Bielova. 2023. Towards a preliminary ontology of dark patterns knowledge. In *Extended abstracts of the 2023 CHI conference on human factors in computing systems*. 1–9.

[28] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[29] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301* (2023).

[30] Hugging Face. 2024. Running models on WebGPU. https://huggingface.co/docs/transformers.js/en/guides/webgpu. Accessed: 2025-04-13.

[31] Rishabh Khandelwal, Thomas Linden, Hamza Harkous, and Kassem Fawaz. 2021. {PriSEC}: A Privacy Settings Enforcement Controller. 465–482. https://www.usenix.org/conference/usenixsecurity21/presentation/khandelwal

[32] Rishabh Khandelwal, Asmit Nayak, Hamza Harkous, and Kassem Fawaz. [n. d.]. Automated Cookie Notice Analysis and Enforcement. ([n. d.]).

[33] J Richard Landis and Gary G Koch. 1977. The measurement of observer agreement for categorical data. *biometrics* (1977), 159–174.

[34] Joffrey L Leevy, Taghi M Khoshgoftaar, Richard A Bauder, and Naeem Seliya. 2018. A survey on addressing high-class imbalance in big data. *Journal of Big Data* 5, 1 (2018), 1–30.

[35] Mark Leiser and Cristiana Santos. 2023. Dark Patterns, Enforcement, and the emerging Digital Design Acquis: Manipulation beneath the Interface. (2023).

[36] Chris Lewis. 2014. Gameful Patterns. In *Irresistible Apps: Motivational Design Patterns for Apps, Games, and Web-based Communities*, Chris Lewis (Ed.). Apress, Berkeley, CA, 33–50. doi:10.1007/978-1-4302-6422-4_4

[37] Ming Li, Ruiyi Zhang, Jian Chen, Jiuxiang Gu, Yufan Zhou, Franck Dernoncourt, Wanrong Zhu, Tianyi Zhou, and Tong Sun. 2025. Towards Visual Text Grounding of Multimodal Large Language Model. *arXiv preprint arXiv:2504.04974* (2025).

[38] Zhangheng Li, Keen You, Haotian Zhang, Di Feng, Harsh Agrawal, Xiujun Li, Mohana Prasad Sathya Moorthy, Jeff Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-ui 2: Mastering universal user interface understanding across platforms. *arXiv preprint arXiv:2410.18967* (2024).

[39] Thomas F. Liu, Mark Craft, Jason Situ, Ersin Yumer, Radomir Mech, and Ranjitha Kumar. 2018. Learning Design Semantics for Mobile Apps. In *The 31st Annual ACM Symposium on User Interface Software and Technology* (Berlin, Germany) *(UIST '18)*. New York, NY, USA, 569–579. doi:10.1145/3242587.3242650

[40] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203* (2024).

[41] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Li Erran

109

Li, Raluca Ada Popa 和 Ion Stoica. 2025. DeepScaleR: 通过扩展 RL 超越 O1-Preview 的 15 亿模型。
https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2. Notion 博客. [42] SM Hasan Mansur, Sabiha Salma, Damilola Awofisayo 和 Kevin Moran. 2023. Aidui: 面向用户界面暗模式自动识别。发表于 2023 年 IEEE/ACM 第 45 届国际软件工程大会 (ICSE). IEEE, 1958–1970. [43] Arunesh Mathur, Gunes Acar, Michael J Friedman, Eli Lucherini, Jonathan Mayer, Marshini Chetty 和 Arvind Narayanan. 2019. 大规模暗模式：来自 11K 个购物网站爬虫的发现。《ACM 人机交互学报》3, CSCW (2019), 1–32. [44] Vijayalakshmi Mohankumar 和 Sasithradevi Anbalagan. [n.d.]. 用于增强水下鱼类检测的基准数据集和集合 YOLO 方法。ETRI 期刊 n/a, n/a ([n.d.]). doi:10.4218/etrij.

Li, Raluca Ada Popa, and Ion Stoica. 2025. DeepScaleR: Surpassing O1-Preview with a 1.5B Model by Scaling RL. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2. Notion Blog.

[42] SM Hasan Mansur, Sabiha Salma, Damilola Awofisayo, and Kevin Moran. 2023. Aidui: Toward automated recognition of dark patterns in user interfaces. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1958–1970.

[43] Arunesh Mathur, Gunes Acar, Michael J Friedman, Eli Lucherini, Jonathan Mayer, Marshini Chetty, and Arvind Narayanan. 2019. Dark patterns at scale: Findings from a crawl of 11K shopping websites. *Proceedings of the ACM on human-computer interaction* 3, CSCW (2019), 1–32.

[44] Vijayalakshmi Mohankumar and Sasithradevi Anbalagan. [n. d.]. A benchmark dataset and ensemble YOLO method for enhanced underwater fish detection. *ETRI Journal* n/a, n/a ([n. d.]). doi:10.4218/etrij.2024-0383 _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.4218/etrij.2024-0383.

[45] Asmit Nayak, Rishabh Khandelwal, Earlence Fernandes, and Kassem Fawaz. 2024. Experimental Security Analysis of Sensitive Data Access by Browser Extensions. In *Proceedings of the ACM on Web Conference 2024*. 1283–1294.

[46] Asmit Nayak, Shirley Zhang, Yash Wani, Rishabh Khandelwal, and Kassem Fawaz. 2024. Automatically Detecting Online Deceptive Patterns. *arXiv preprint arXiv:2411.07441* (2024).

[47] notAI tech. 2019. NudeNet: Nudity detection with deep neural networks. https://github.com/notAI-tech/NudeNet. Accessed March 19, 2025.

[48] OpenAI. [n. d.]. Learning to reason with LLMs. https://openai.com/index/learning-to-reason-with-llms/. https://openai.com/index/learning-to-reason-with-llms/

[49] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[50] Vung Pham, Lan Dong Thi Ngoc, and Duy-Linh Bui. 2024. Optimizing YOLO Architectures for Optimal Road Damage Detection and Classification: A Comparative Study from YOLOv7 to YOLOv10. doi:10.48550/arXiv.2410.08409 arXiv:2410.08409 [cs].

[51] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. 2018. Tranco: A research-oriented top sites ranking hardened against manipulation. *arXiv preprint arXiv:1806.01156* (2018).

[52] S Hrushikesava Raju, Saiyed Faiayaz Waris, S Adinarayna, Vijaya Chandra Jadala, and G Subba Rao. 2022. Smart dark pattern detection: Making aware of misleading patterns through the intended app. In *Sentimental Analysis and Deep Learning: Proceedings of ICSADL 2021*. Springer, 933–947.

[53] Joseph Redmon. 2018. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767* (2018).

[54] Shaoqing Ren. 2015. Faster r-cnn: Towards real-time object detection with region proposal networks. *arXiv preprint arXiv:1506.01497* (2015).

[55] Matthew Renze. 2024. The effect of sampling temperature on problem solving in large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 7346–7356.

[56] Yasin Sazid, Mridha Md Nafis Fuad, and Kazi Sakib. 2023. Automated Detection of Dark Patterns Using In-Context Learning Capabilities of GPT-3. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 569–573.

[57] René Schäfer, Paul Miles Preuschoff, Rene Niewianda, Sophie Hahn, Kevin Fiedler, and Jan Borchers. 2025. Don't Detect, Just Correct: Can LLMs Defuse Deceptive Patterns Directly?. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–11.

[58] Schneider Wallace. 2023. Dark Patterns: Making Online Subscriptions Harder to Cancel Draws Lawsuits, Settlements, and Government Scrutiny. https://www.schneiderwallace.com/media/dark-patterns-making-online-subscriptions-harder-to-cancel-draws-lawsuits-settlements-and-government-scrutiny/.

[59] Vinoth Pandian Sermuga Pandian, Sarah Suleri, and Prof Dr Matthias Jarke. 2021. UISketch: a large-scale dataset of UI element sketches. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–14.

[60] Zewei Shi, Ruoxi Sun, Jieshan Chen, Jiamou Sun, Minhui Xue, Yansong Gao, Feng Liu, and Xingliang Yuan. 2025. 50 Shades of Deceptive Patterns: A Unified Taxonomy, Multimodal Detection, and Security Implications. In *Proceedings of the ACM Web Conference 2025 (WWW'25)*.

[61] Fatemeh Shiri, Xiao-Yu Guo, Mona Golestan Far, Xin Yu, Gholamreza Haffari, and Yuan-Fang Li. 2024. An Empirical Analysis on Spatial Reasoning Capabilities of Large Multimodal Models. *arXiv preprint arXiv:2411.06048* (2024).

[62] Sina Shool, Sara Adimi, Reza Saboori Amleshi, Ehsan Bitaraf, Reza Golpira, and Mahmood Tara. 2025. A systematic review of large language model (LLM) evaluations in clinical medicine. *BMC Medical Informatics and Decision Making* 25, 1 (2025), 117.

[63] Nakatani Shuyo and others (ported to Python). 2021. langdetect: Language detection library ported to Python. https://pypi.org/project/langdetect/. Version [Specify version if needed, e.g., 1.0.9].

110

[64] Than Htut Soe, Oda Elise Nordberg, Frode Guribye 和 Marija Slavkovik. 2020. 设计规避——在线新闻网站 Cookie 同意中的暗模式。在《第十一届北欧人机交互会议论文集：塑造体验，塑造社会》中。1–12。[65] Luke Stein. 2019. Luke Stein 在 X 上的推文。https://x.com/lukestein/status/1150014732486742016。访问日期：2024-09-02。[66] Alina Stöver, Nina Gerber, Henning Pridöhl, Max Maass, Sebastian Bretthauer, Indra Spiecker Gen. Döhmann, Matthias Hollick 和 Dominik Herrmann. 2023. 网站所有者如何应对隐私问题：通过一次隐蔽通知研究的回应主题分析揭示了多样的情况和挑战。《隐私增强技术论文集》2023, 2 (2023年4月), 251–264. doi: 10 56553 2023 0051。 /popets- - Jiejun Tan, Zhicheng Dou, [67] Wen Wang, Mang Wang, Weipeng Chen 和 Ji-Rong Wen. 2025. Htmlrag: HTML 比纯文本更适合在 RAG 系统中建模检索到的知识。在《会议论文集》中

[64] Than Htut Soe, Oda Elise Nordberg, Frode Guribye, and Marija Slavkovik. 2020. Circumvention by design-dark patterns in cookie consent for online news outlets. In *Proceedings of the 11th nordic conference on human-computer interaction: Shaping experiences, shaping society*. 1–12.

[65] Luke Stein. 2019. Tweet by Luke Stein on X. https://x.com/lukestein/status/1150014732486742016. Accessed: 2024-09-02.

[66] Alina Stöver, Nina Gerber, Henning Pridöhl, Max Maass, Sebastian Bretthauer, Indra Spiecker Gen. Döhmann, Matthias Hollick, and Dominik Herrmann. 2023. How Website Owners Face Privacy Issues: Thematic Analysis of Responses from a Covert Notification Study Reveals Diverse Circumstances and Challenges. *Proceedings on Privacy Enhancing Technologies* 2023, 2 (April 2023), 251–264. doi:10.56553/popets-2023-0051

[67] Jiejun Tan, Zhicheng Dou, Wen Wang, Mang Wang, Weipeng Chen, and Ji-Rong Wen. 2025. Htmlrag: Html is better than plain text for modeling retrieved knowledge in rag systems. In *Proceedings of the ACM on Web Conference 2025*. 1733–1746.

[68] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[69] Maxim Tkachenko, Mikhail Malyuk, Andrey Holmanyuk, and Nikolai Liubimov. 2020-2025. Label Studio: Data labeling software. https://github.com/HumanSignal/label-studio Open source software available from https://github.com/HumanSignal/label-studio.

[70] Rahee Walambe, Aboli Marathe, Ketan Kotecha, and George Ghinea. 2021. Lightweight Object Detection Ensemble Framework for Autonomous Vehicles in Challenging Weather Conditions. *Computational Intelligence and Neuroscience* 2021 (Oct. 2021), 5278820. doi:10.1155/2021/5278820

[71] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. 2024. Yolov10: Real-time end-to-end object detection. *arXiv preprint arXiv:2405.14458* (2024).

[72] David Wang, John Anthony Ribando, David Mo, and Nicholas Tung. 2019. *insite. A chrome extension that protects consumers from marketing tricks when they shop online.* Retrieved 6 Sep 2022 from https://devpost.com/software/insite-qfpjcd

[73] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[74] Jason Wu, Rebecca Krosnick, Eldon Schoop, Amanda Swearngin, Jeffrey P. Bigham, and Jeffrey Nichols. 2023. Never-ending Learning of User Interfaces. doi:10.48550/arXiv.2308.08726 arXiv:2308.08726 [cs].

[75] Jason Wu, Siyan Wang, Siman Shen, Yi-Hao Peng, Jeffrey Nichols, and Jeffrey Bigham. 2023. WebUI: A Dataset for Enhancing Visual UI Understanding with Web Semantics. *ACM Conference on Human Factors in Computing Systems (CHI)* (2023).

[76] Mulong Xie, Sidong Feng, Zhenchang Xing, Jieshan Chen, and Chunyang Chen. 2020. UIED: a hybrid tool for GUI element detection. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1655–1659.

[77] Mulong Xie, Sidong Feng, Zhenchang Xing, Jieshan Chen, and Chunyang Chen. 2020. UIED: a hybrid tool for GUI element detection. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) *(ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1655–1659. doi:10.1145/3368089.3417940

[78] Haoxuan You, Haotian Zhang, Zhe Gan, Xianzhi Du, Bowen Zhang, Zirui Wang, Liangliang Cao, Shih-Fu Chang, and Yinfei Yang. 2023. Ferret: Refer and ground anything anywhere at any granularity. *arXiv preprint arXiv:2310.07704* (2023).

[79] Yuhang Zang, Wei Li, Jun Han, Kaiyang Zhou, and Chen Change Loy. 2025. Contextual object detection with multimodal large language models. *International Journal of Computer Vision* 133, 2 (2025), 825–843.

[80] Xiaoyi Zhang, Lilian De Greef, Amanda Swearngin, Samuel White, Kyle Murray, Lisa Yu, Qi Shan, Jeffrey Nichols, Jason Wu, Chris Fleizach, et al. 2021. Screen recognition: Creating accessibility metadata for mobile applications from pixels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–15.

[81] Yuhui Zhang, Alyssa Unell, Xiaohan Wang, Dhruba Ghosh, Yuchang Su, Ludwig Schmidt, and Serena Yeung-Levy. 2024. Why are visually-grounded language models bad at image classification? *arXiv preprint arXiv:2405.18415* (2024).

[82] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911* (2023).

110